Klasifikacija karaktera

Nemanja Kelecevic

March 9, 2026

1 Uvod

1.1 Definicija zadatka

Cilj projekta je izgradnja i evaluacija sistema za klasifikaciju karaktera na osnovu njihove vizuelne reprezentacije u formi slike dimenzije 20x20 piksela. Svaka instanca predstavlja jedan karakter opisan skupom od 400 numerickih atributa koji odgovaraju vrednostima piksela, dok je klasna oznaka definisana 8-bitnim kodom karaktera (256 mogucih klasa).

Osnovni zadatak projekta je razvoj i poredjenje vise algoritama klasifikacije koji, na osnovu sirovih piksel vrednosti, predvidjaju odgovarajuci karakter. Pored primarnog cilja, projekat ukljucuje i analizu uticaja fonta i stilskih varijacija (strength i italic) na performanse modela, kako bi se ispitao stepen robustnosti sistema na promene vizuelnog izgleda karaktera. Sprovedene su i finije analize pojedinacnih modela kako bi se detaljnije upoznali sa strukturom skupa podataka i karakteristikama modela koji pocivaju na razlicitim principima.

Problem prepoznavanja karaktera (OCR – Optical Character Recognition) predstavlja klasicnu oblast primene masinskog ucenja. Iako savremeni sistemi koriste neuronske mreze i kompleksne arhitekture, osnovni klasifikacioni pristup nad piksel reprezentacijom ostaje fundamentalan model za razumevanje ponasanja algoritama.

Ovaj projekat omogucava analizu kako razliciti algoritmi reaguju na visedimenzionalne podatke, koliko su modeli osetljivi na promene fonta i stilskih osobina, kao i u kojoj meri vizuelne varijacije uticu na tacnost klasifikacije.

1.2 Ideja realizacije

Realizacija projekta zasnovana je na pristupu koji obuhvata nekoliko kljucnih koraka.

Najpre se konstruise skup podataka spreman za modelovanje, uz obezbedjivanje metodoloski ispravne podele na trening i test skup. Ovakav pristup omogucava fer poredjenje razlicitih algoritama klasifikacije, jer svi modeli koriste identican train/test split.

Zatim se implementira vise algoritama klasifikacije, kao sto su logisticka regresija, SVM, Random Forest i Gradient Boosting. Za svaki model koristi se odgovarajuca pipeline struktura, ukljucujuci standardizaciju podataka kada je to potrebno. Time se obezbedjuje konzistentnan oblik modela koji su spremni za visestruku upotrebu.

Performanse modela evaluiraju se na test skupu uz pomoc standardnih metrika kao sto su accuracy i matrica konfuzije. Posebna paznja posvecuje se uporednoj analizi dobijenih rezultata, kako bi se identifikovale razlike u ponasanju algoritama u odnosu na kompleksnost problema i broj klasa.

Dodatni i znacajan deo projekta predstavlja analiza robustnosti modela. Nakon treniranja klasifikatora, performanse se analiziraju po pojedinacnim fontovima, kao i u odnosu na stil karaktera (italic) i parametar strength. Ovakva analiza omogucava dublje razumevanje u kojim situacijama modeli ostvaruju slabije rezultate i koliko su osetljivi na vizuelne varijacije.

Na taj nacin projekat kombinuje standardni klasifikacioni pristup sa detaljnom analizom faktora koji uticu na performanse, cime se dobija sveobuhvatna analiza ponasanja algoritama u problemu prepoznavanja karaktera.

2 Ucitavanje i analiza skupa podataka

2.1 Organizacija fajlova

Dataset je organizovan kao skup vise CSV fajlova. Svaki CSV fajl odgovara jednom fontu (npr. `ARIAL.csv`, `AGENCY.csv`, itd.). To znaci da svi redovi u okviru jednog CSV-a predstavljaju uzorke karaktera generisane ili skenirane u istom fontu. Ovakva podela omogucava da se podaci ucitavaju parcijalno (po fontu), ili da se po potrebi spoje u jedan veci skup.

2.2 Sta predstavlja jedna instanca

Svaki red u CSV tabeli predstavlja jednu instancu: sliku jednog karaktera, normalizovanu na fiksnu rezoluciju 20x20 piksela. Pored same slike, u redu postoje i prateci atributi koji opisuju font i nacin nastanka uzorka (npr. varijanta fonta, italic, jacina otiska, originalne dimenzije pre obrade, itd.). Drugim recima, jedna instanca se moze opisati kao: "Karakter X (labela) prikazan u konkretnom fontu i stilu, gde su pikseli slike upisani kao vrednosti u kolonama r00 - r19."

2.3 Atributi u tabeli: meta-kolone i piksel-kolone

Meta-kolone nose informacije o uzorku koje nisu direktno pikseli slike. U ucitanom primeru vide se tipicne meta-kolone kao sto su:

- `font`: naziv fonta (npr. ARIAL).
- `fontVariant`: izvor uzorka; u praksi se moze pojaviti vrednost poput "scanned", sto ukazuje da je uzorak dobijen skeniranjem.
- `m_label`: numericka oznaka karaktera (klasa). U praksi se ova vrednost moze interpretirati kao kod karaktera (cesto ASCII/Unicode), sto omogucava dekodiranje u konkretan znak.
- `italic`: indikator da li je karakter ispisan u italic stilu (0 ili 1).
- `strength`: parametar koji opisuje da li je font boldiran ili ne.
- `orientation`: parametar orijentacije/rotacije.
- `m_top`, `m_left`: tehnicke koordinate vezane za poziciju gornjeg/najlevljeg dela karaktera u originalnoj slici pre normalizacije.
- `originalH`, `originalW`: originalne dimenzije pre obrade (pre nego sto je uzorak sveden na 20x20).

Ove kolone su korisne za analizu i dodatne eksperimente (npr. klasifikacija fonta ili italic stila), ali u osnovnom scenariju cilj je da se prepozna karakter iz same slike, pa se kao ulazne osobine najcesce koriste samo pikseli.

Glavni deo podataka cine kolone koje predstavljaju piksele slike 20x20. Kolone su imenovane u formatu `rIcJ`, gde je I indeks reda piskela a j indeks kolone piksela. Na primer, kolona `r19c11` predstavlja piksel u 20. redu i 12. koloni. Ukupno ima 20x20=400 ovakvih kolona.

Vrednosti u ovim kolonama su brojevi u opsegu 1-255 i predstavljaju intenzitet sivila (grayscale) za dati piksel. Prakticno, to znaci da slike nisu RGB, vec su kodirane jednom vrednoscu po pikselu. U posmatranom primeru, uocava se da vrednost 255 odgovara svetloj pozadini, dok manje vrednosti predstavljaju tamnije delove slova, a vrednosti izmedju predstavljaju prelazne nijanse nastale antialiasing-om.

2.4 Rekonstrukcija slike iz tabele

Da bismo potvrdili da kolone `rIcJ` zaista predstavljaju piksele slike, sproveden je postupak rekonstrukcije slike iz jednog reda tabele: izabrao sam jedan red iz data frame-a, izdvojio sve piksel kolone i iscrtao ih. Tokom prikaza uoceno je da je pozadina uglavnom svetla (255), a karakter tamniji (manje vrednosti). Radi bolje vizualne interpretacije, invertovao sam matricu pre samog iscrtavanja kako bih dobio efekat tamnog slova na svetloj pozadini - intuitivniji prikaz.
Kao dodatni dokaz tumacenja iznad, prikazano je vise primera (npr. 10 uzoraka) rasporedjenih u mrezi subplotova (2x5). Za svaki uzorak prikazana je slika i naslov u obliku `char (label)`. Ovaj prikaz jasno pokazuje da iz tablicnih vrednosti dobijamo prepoznatljive oblike karaktera, sto potvrdjuje korektno tumacenje strukture dataseta.
U projektu se nalazi fajl preprocessing/01_load_preview.ipynb u kojem je ceo proces kodiran.

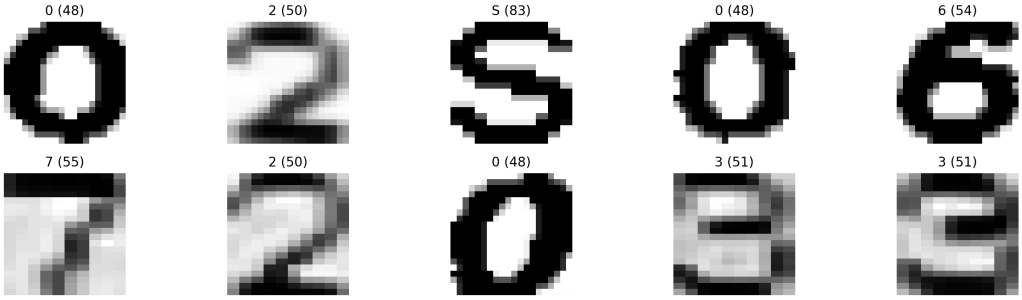


Figure 1: Primer rekonstrukcije karaktera iz 20x20 piksel-kolona. Naslov svake slike prikazuje dekodirani karakter i njegovu numericku labelu.

3 Preprocesiranje

Preprocesiranje podataka predstavlja kljucni korak u pripremi skupa za treniranje i evaluaciju klasifikacionih modela. Cilj ovog koraka bio je da se obezbedi memorijski efikasna, metodoloski ispravna i reproduktivna obrada velikog skupa podataka pre faze modelovanja. Posebna paznja posvecena je optimizaciji tipova podataka, kontroli zauzeca memorije i pravilnoj organizaciji izlaznog formata.

3.1 Formiranje skupa podataka

Skup podataka sastoji se od 153 CSV fajla ukupne velicine priblizno 1.1 GB. Svaka instanca opisana je sa 400 atributa koji predstavljaju piksele slike dimenzije 20×20, kao i klasnom oznakom *m_label*. Nakon agregacije svih fajlova dobijeno je ukupno 832 670 instanci. Ukupan broj piksel vrednosti u skupu iznosi:

$$832\,670 \times 400 = 333\,068\,000$$

pojedinacnih numerickih vrednosti.

Tokom ucitavanja podaci su privremeno skladisteni u Python listama, a tek nakon toga objedinjeni u jednu NumPy matricu. Ovakav pristup izabran je zbog toga sto NumPy nizovi imaju fiksnu velicinu i svako prosirivanje zahteva realokaciju memorije i kopiranje svih prethodnih podataka. Koriscenjem liste izbegava se kvadratna vremenska slozenost i nepotrebno povecanje maksimalnog trenutnog zauzeca memorije (tzv. peak memory).

3.2 Nedostajuce vrednosti

Jedan od osnovnih koraka tokom faze preprocesiranja jeste provera prisustva nedostajucih vrednosti u skupu podataka. Prisustvo *NaN* vrednosti moze negativno uticati na proces treniranja modela, jer veliki broj algoritama masinskog ucenja ne podrzava rad sa nedostajucim podacima bez dodatne obrade.

Analizom svih relevantnih komponenti skupa podataka utvrdjeno je da matrica ulaznih atributa X, kao i vektori klasnih oznaka y i pomocnog atributa *strength*, ne sadrze nedostajuce vrednosti. Drugim recima, svaka instanca u skupu podataka je kompletno definisana, bez praznih ili nedefinisanih atributa.

S obzirom na to da nedostajuce vrednosti nisu detektovane, nije bilo potrebe za primenom dodatnih tehnika obrade poput imputacije ili uklanjanja instanci.

3.3 Elementi van granica

Pored provere nedostajucih vrednosti, izvrsena je i analiza moguceg prisustva elemenata van ocekivanog opsega vrednosti. Ovakvi elementi (*outliers*) mogu predstavljati posledicu gresaka tokom generisanja ili ucitavanja podataka i mogu negativno uticati na proces ucenja modela.

U konkretnom skupu podataka svako obelezje predstavlja intenzitet piksela slike karaktera dimenzije 20×20. Intenziteti piksela su kodirani celobrojnim vrednostima u opsegu $[0, 255]$, sto odgovara standardnoj reprezentaciji sivih tonova u 8-bitnom formatu slike.

Analizom matrice obelezja utvrdjeno je da minimalna vrednost piksela iznosi 0, dok maksimalna vrednost iznosi 255. Time je potvrdeno da sve vrednosti pripadaju ocekivanom intervalu, bez prisustva elemenata van granica.

Na osnovu toga zakljucuje se da skup podataka ne sadrzi anomalne vrednosti koje bi zahtevale dodatnu obradu ili filtriranje tokom faze preprocesiranja.

3.4 Balansiranost klasa

Radi procene balansiranosti skupa podataka analizirana je raspodela broja instanci po klasama. Na slici je prikazana distribucija uzoraka za svih 256 klasa, gde horizontalna osa predstavlja indeks klase, dok vertikalna osa predstavlja broj instanci koje pripadaju datoj klasi.

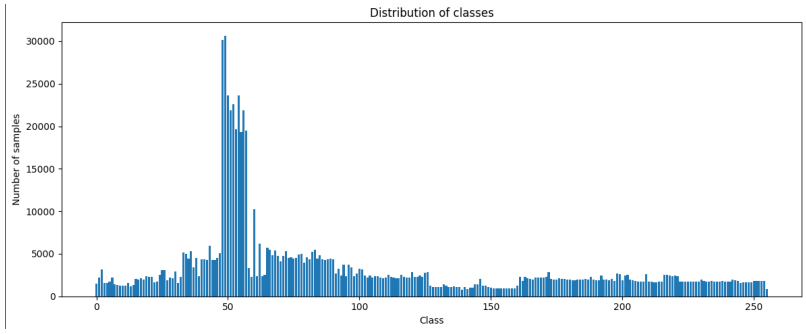


Figure 2: Balansiranost klasa

Sa grafa se moze uociti da je vecina klasa zastupljena sa relativno slicnim brojem instanci, sto ukazuje na umereno balansiran skup podataka. Ipak, primecuje se izrazito povecana zastupljenost odredjenog opsega klasa, otprilike izmedju vrednosti 45 i 60.

Ovaj opseg odgovara ASCII kodovima cifara (karakteri '0'–'9'), sto objasnjava zasto su upravo te klase znacajno zastupljenije u odnosu na ostale. Ovakva raspodela je ocekivana, jer se cifre u realnim tekstualnim podacima pojavljuju relativno cesto.

Veca zastupljenost ovih klasa implicira da ce modeli tokom procesa ucenja imati vise informacija o njihovim karakteristikama. Posledicno, moze se ocekivati da ce upravo cifre biti medju najuspesnije klasifikovanim karakterima, sto je kasnije potvrdjeno i eksperimentalnim rezultatima.

3.5 Dodatni atributi skupa podataka

Pored matrice ulaznih atributa koja opisuje piksele slike, skup podataka sadrzi i nekoliko dodatnih atributa koji karakterisu vizuelne osobine generisanih karaktera.

Utvrdjeno je da skup podataka sadrzi 153 razlicita fonta, cime je obezbedjena znacajna tipografska raznovrsnost karaktera. Pored toga, atribut *italic* predstavlja binarnu informaciju o tome da li je karakter prikazan u iskosenoj verziji, pri cemu su moguce vrednosti 0 i 1.

Dodatni atribut *strength* opisuje debljinu poteza karaktera i poprima vrednosti u opsegu $[0.4, 0.7]$. Varijacije ovog parametra dodatno povecavaju vizuelnu raznolikost uzoraka u skupu podataka. Zanimljivo je primetiti da su vrednosti samo u ovom malom opsegu, pretpostavka je da ta debljina najfinije ispisuje karakter. Manja debljina bi dala bledu, nedovoljno jasnu sliku a veca bi spojila ivice "uskih" karaktera.

Prisustvo ovih faktora doprinosi vecoj kompleksnosti problema klasifikacije, jer isti karakter moze biti prikazan u razlicitim fontovima, stilovima i debljinama poteza.

3.6 Optimizacija tipova podataka i analiza zauzeca memorije

Pikseli su tokom ucitavanja konvertovani u tip `uint8`, jer vrednosti pripadaju opsegu $[0, 255]$. Tip `uint8` zauzima 1 bajt po vrednosti, dok podrazumevani tip `int64` zauzima 8 bajtova. Ukupno zauzece memorije u slucaju `uint8` iznosi:

$$333\,068\,000 \times 1 = 333\,068\,000B \approx 318MB$$

Ukoliko bi se koristio `int64`, zauzece bi bilo:

$$333\,068\,000 \times 8 = 2\,664\,544\,000B \approx 2.48GB$$

Razlika iznosi preko 2 GB memorije, pa je konverzija u tip uint8 donela ogroman znacaj kada je memorijska efikasnost daljeg rada nad podacima u pitanju. Nakon agregacije i validacije podataka, ceo skup podataka konvertovan je u tip `float32`, koji zauzima 4 bajta po vrednosti i predstavlja optimalan kompromis izmedju numericke preciznosti i memorijske efikasnosti. Zauzece u tom slucaju iznosi:

$$333\,068\,000 \times 4 = 1\,332\,272\,000B \approx 1.24GB$$

Konverzija je izvrsena tek nakon objedinjavanja podataka, cime je kontrolisano maksimalno trenutno zauzece memorije.

3.7 Ukljucivanje dodatnih atributa za analizu

Pored matrice sa atributima X i klasne oznake y, paralelno su sacuvane i informacije o fontu, parametru `strength` i indikatoru `italic`. Ove promenljive nisu koriscene kao ulaz modela za klasifikaciju karaktera, vec iskljucivo za analizu robustnosti sistema - kako se model ponasa na razlicitim podskupovima citavog skupa podataka. Na ovaj nacin omoguceno je kasnije grupisanje performansi modela po pojedinacnim fontovima, kao i analiza uticaja stilskih varijacija na tacnost klasifikacije, bez narusavanja metodoloske cistoce osnovnog zadatka.

3.8 Podela na trening i test skup

Podaci su podeljeni na trening i test skup u odnosu 80:20, uz primenu stratifikacije po klasnoj oznaci. Stratifikacija obezbedjuje da distribucija svih 256 klasa ostane proporcionalna u oba podskupa, cime se osigurava fer i pouzdana evaluacija modela.

3.9 Cuvanje preprocesiranog skupa podataka

Preprocesirani podaci sacuvani su u binarnom formatu `.npz` pomocu funkcije `np.savez_compressed`. Format `.npz` predstavlja kompresovani ZIP kontejner koji omogucava cuvanje vise NumPy nizova u jednom fajlu, uz zadrzavanje informacije o tipu podataka.

Ukoliko bi se podaci cuvali u CSV formatu, svaka numericka vrednost bila bi zapisana kao tekst. Na primer, vrednost tipa `float32` (4 bajta) u CSV fajlu moze zauzimati 6–12 karaktera, odnosno 6–12 bajtova. U najjednostavnijem slucaju, zauzece bi bilo:

$$333\,068\,000 \times 8 \approx 2.66GB$$

sto je vise nego duplo u odnosu na binarnu reprezentaciju.

Binarni `.npz` format omogucava direktno ucitavanje memorijskih blokova bez parsiranja tekstualnog sadrzaja, cime se znacajno ubrzava rad sistema i smanjuje velicina fajla na disku. Dodatna kompresija dodatno redukuje prostor bez gubitka informacija.

Ovim pristupom obezbedjeno je jasno razdvajanje obrade podataka od faze modeliranja, sto doprinosi modularnosti, preglednosti i reproduktivnosti celokupnog projekta.

3.10 PCA redukcija dimenzionalnosti

U okviru ovog projekta primenjena je metoda *Principal Component Analysis (PCA)* sa ciljem smanjenja dimenzionalnosti prostora atributa. PCA je jedna od najčešće korišćenih tehnika za redukciju dimenzionalnosti u mašinskom učenju i statistici. Osnovna ideja ove metode je da se originalni skup atributa transformiše u novi skup koordinata (tzv. *principal components*) koje su medjusobno nekorelirane i uredjene prema količini varijanse koju objašnjavaju u podacima. Drugim rečima, PCA pokušava da pronadje pravce u prostoru podataka u kojima se nalazi najveća količina informacija.

Intuitivno posmatrano, PCA vrši rotaciju koordinatnog sistema tako da se podaci projektuju na nove ose koje bolje opisuju njihovu strukturu. Prva glavna komponenta predstavlja pravac u kojem je varijansa podataka najveća, druga komponenta predstavlja sledeći najvažniji pravac koji

je ortogonalan na prvi, i tako dalje. Zadržavanjem samo prvih nekoliko komponenti moguće je zadržati većinu informacija iz originalnih podataka, dok se ostatak (koji često predstavlja šum ili redundantne informacije) može odbaciti.

U našem slučaju originalni skup podataka sadrži 400 numeričkih atributa koji predstavljaju piksel vrednosti slika karaktera. Veliki broj atributa može predstavljati problem iz više razloga. Prvo, neki atributi mogu biti medjsobno visoko korelisani, što znači da zapravo nose sličnu informaciju. Drugo, visoka dimenzionalnost može povećati složenost modela i vreme treniranja, naročito kod algoritama koji su osetljivi na broj dimenzija, kao što su logistička regresija ili SVM. Zbog toga je primenjena PCA metoda kako bi se prostor atributa smanjio uz očuvanje većine informacija sadržanih u podacima.

Postupak primene PCA

Primena PCA metode sprovedena je kroz nekoliko koraka. Prvi korak je bilo **skaliranje podataka**. PCA je metoda koja se zasniva na analizi varijanse atributa, pa je veoma važno da svi atributi budu na istoj skali. Ukoliko bi neki atributi imali značajno veće numeričke vrednosti od drugih, oni bi dominirali u procesu odredjivanja glavnih komponenti. Zbog toga je korišćen `StandardScaler`, koji transformiše svaki atribut tako da ima srednju vrednost 0 i standardnu devijaciju 1. Ova transformacija je naučena na trening skupu (`fit` operacija), a zatim primenjena i na trening i na test skup (`transform`). Na taj način se izbegava tzv. *data leakage*, odnosno situacija u kojoj bi informacije iz test skupa uticale na proces treniranja modela.

Nakon skaliranja podataka primenjena je PCA transformacija. U početnoj fazi PCA je treniran bez ograničenja broja komponenti kako bi se analizirala raspodela objašnjene varijanse po komponentama. Nakon treniranja PCA modela izračunata je **kumulativna objašnjena varijansa** (*cumulative explained variance*), koja pokazuje koliki procenat ukupne varijanse podataka je objašnjen sa prvih k komponenti. Na osnovu ovih vrednosti nacrtan je graf kumulativne objašnjene varijanse u zavisnosti od broja komponenti.

Analizom ovog grafa može se videti kako se količina objašnjene varijanse povećava sa brojem komponenti. U početku graf raste veoma brzo, što znači da prvih nekoliko komponenti nosi veliki deo informacija iz podataka. Kako se broj komponenti povećava, rast krive postaje sporiji, što ukazuje da dodatne komponente donose sve manju količinu nove informacije.

Na osnovu ovog grafa odredjeno je koliko komponenti je potrebno da bi se zadržao veliki deo varijanse podataka. Konkretno, pronadjen je indeks prve komponente za koju kumulativna objašnjena varijansa prelazi 95% ukupne varijanse. Dobijeno je da je za dostizanje ovog praga potrebno 119 glavnih komponenti.

Nakon toga je konstruisan finalni PCA model sa parametrom `n_components = 119`. Ovaj model je treniran nad skaliranim trening podacima, a zatim je transformacija primenjena i na trening i na test skup.

Rezultat redukcije dimenzionalnosti

Primena PCA metode omogućila je značajno smanjenje dimenzionalnosti skupa podataka. Originalni skup podataka dimenzionalnosti $(666136, 400)$ redukovan je na prostor dimenzionalnosti $(666136, 119)$, pri čemu je zadržano približno 95% ukupne varijanse podataka. Time je broj atributa smanjen za više od tri puta uz minimalan gubitak informacija.

4 Logisticka regresija

4.1 Koncept

Logisticka regresija je jedan od najpoznatijih i najcesce koriscenih algoritama za probleme klasifikacije. Osnovna ideja algoritma je da se iz ulaznih atributa izracuna verovatnoca pripadnosti svakoj klasi, a zatim se primer klasifikuje u onu klasu koja ima najvecu predvidjenu verovatnocu.

Algoritam se zasniva na linearnoj kombinaciji ulaznih atributa. Za svaki ulazni primer model racuna izraz oblika

$$z = w^T x + b \tag{1}$$

gde je x vektor ulaznih karakteristika, w vektor tezina modela, a b bias parametar.

Primetimo da je ova funkcija zastupljena i u drugim modelima kao sto su neuronske mreze ili SVM. I ovi modeli koriste slican oblik linearne funkcije kao osnovu zakljucivanja, dok se razlikuju u nacinu na koji se ta funkcija formira i kako se njeni parametri uce tokom treninga.

Ovaj izraz predstavlja linearni model koji opisuje kako ulazne karakteristike uticu na odluku modela. Medjutim, posto klasifikacija zahteva verovatnoce u opsegu od 0 do 1, linearni izlaz se transformise pomocu logisticke (sigmoidne) funkcije

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

Sigmoidna funkcija preslikava realne vrednosti u interval $[0, 1]$, sto omogucava interpretaciju izlaza modela kao verovatnoce pripadnosti odredjenoj klasi. Na taj nacin model ne daje samo odluku o klasi, vec i procenu koliko je siguran u tu odluku.

U ovom projektu problem nije binarna klasifikacija, vec klasifikacija velikog broja karaktera. Zbog toga koristimo **multiklasnu logisticku regresiju**, gde model procenjuje verovatnocu za svaku mogucu klasu karaktera. Za svaki ulazni primer izracunavaju se verovatnoce svih klasa, a zatim se kao rezultat uzima klasa sa najvecom verovatnocom.

Da bi model naucio odgovarajuce vrednosti tezina, koristi se **funkcija gubitka** (*loss function*) koja meri koliko su predvidjanja modela udaljena od stvarnih oznaka u trening skupu. Kod logisticke regresije standardna funkcija gubitka je **log-loss**, poznata i kao **cross-entropy loss**. Ona kaznjava situacije u kojima model daje veliku verovatnocu pogresnoj klasi, a nagradjuje situacije kada je verovatnoca tacne klase visoka. Tokom treninga algoritam pokusava da **minimizuje ovu funkciju gubitka**, odnosno da nadje takve vrednosti tezina koje daju sto tacnija predvidjanja.

U kontekstu ovog projekta logisticka regresija se koristi za klasifikaciju karaktera generisanih iz razlicitih fontova. Svaki karakter je predstavljen nizom numerickih karakteristika koje opisuju njegov oblik, odnosno raspored piksela. Ove karakteristike predstavljaju ulaz modela. Tokom treninga model uci koje kombinacije karakteristika odgovaraju kojim karakterima, odnosno pokusava da nauci granice izmedju klasa u prostoru atributa.

Kada se model istrenira, za svaki novi primer (karakter iz test skupa) racuna se verovatnoca pripadnosti svakoj klasi. Kod vise klasa koristi se **softmax funkcija**

$$P(y = k \mid x) = \frac{e^{w_k^T x}}{\sum_{j=1}^{K} e^{w_j^T x}} \tag{3}$$

Klasa sa najvecom verovatnocom zatim se uzima kao predikcija modela. Drugim recima, logisticka regresija predstavlja najjednostavniji model za klasifikaciju, dok slozeniji modeli poput **SVM-a** uvode ideju optimalne granice izmedju klasa, a **neuronske mreze** omogucavaju ucenje znatno kompleksnijih i nelinearnih struktura u podacima. Upravo zbog te jednostavnosti logisticka regresija se cesto koristi kao **bazni model (baseline)** u problemima klasifikacije. Ako i ovako jednostavan model daje dobru tacnost, to znaci da su obrasci u podacima relativno jasni i linearno razdvojivi. U suprotnom, potrebno je koristiti slozenije modele kao sto su **SVM sa kernelom** ili **neuronske mreze**.

4.2 Implementacija

4.2.1 Organizacija projekta

Nakon pripreme i preprocesiranja dataset-a, sledeci korak u projektu bio je treniranje i analiza modela logisticke regresije. Implementacija: src/models/logistic_regression.ipynb, predstavlja centralno mesto gde se vrsi treniranje modela, evaluacija performansi i skladistenje rezultata.

Projekat je organizovan tako da se u direktorijumu `src/models` treniraju modeli razlicitih algoritama, vrse njihove predikcije i generisu analize rezultata (vise o njima kasnije), koje se zatim cuvaju u `results/` direktorijumu. Ideja ove organizacije bila je da se sve analize modela nalaze na jednom mestu, kako bi njihovo kasnije poredjenje bilo jednostavnije i preglednije.

Takodje, u `src` direktorijumu nalaze se i implementacije pomocnih klasa opste namene, poput funkcija za ucitavanje podataka i skladistenje rezultata analiza modela.

4.2.2 Ucitavanje preprocesiranog dataset-a

Prvi korak u samom notebook-u jeste ucitavanje preprocesiranog dataset-a. Dataset je prethodno pripremljen u fazi preprocesiranja i sacuvan u `.npz` formatu, nakon cega se prilikom pokretanja notebook-a raspakuju svi nizovi koji su sacuvani, kako bi mogli da se koriste za treniranje modela i kasniju analizu rezultata.

Podaci su vec u fazi preprocesiranja podeljeni na **trening** i **test** skup, kako bi svi modeli u projektu radili nad istim podacima i kako bi njihovo kasnije poredjenje imalo smisla.

Pored osnovnih atributa koji predstavljaju ulaz u model, ucitavaju se i dodatni **meta atributi** koji opisuju karaktere, kao sto su **font**, **italic stil** i **debljina karaktera**. Ove informacije nisu direktan ulaz modelu tokom treniranja, ali su veoma korisne za kasniju analizu performansi modela. Na osnovu njih moguce je analizirati kako razlicite karakteristike fonta uticu na uspeh klasifikacije i gde model najcesce pravi greske.

4.2.3 Konstrukcija modela

Nakon ucitavanja podataka definise se model koji ce se koristiti za klasifikaciju. U implementaciji je koriscen `Pipeline` objekat iz biblioteke *scikit-learn*. Pipeline omogucava da se vise koraka obrade podataka i treniranja modela objedini u jednu celinu.
Model logisticke regresije implementiran je pomocu klase `LogisticRegression`. U okviru pipeline-a primenjuju se dva osnovna koraka. Prvi korak je **standardizacija podataka** pomocu transformacije `StandardScaler`. Standardizacija transformise svaku karakteristiku tako da ima srednju vrednost 0 i standardnu devijaciju 1. Ovaj korak je vazan jer logisticka regresija koristi optimizacione metode koje su osetljive na razliku u skalama atributa. Iako su vrednosti atributa piksela u istom rasponu ($[0, 255]$), moguce je da pojedini pikseli imaju neravnomernu distribuciju vrednosti, pa njihove srednje vrednosti i standardne devijacije mogu znacajno da se razlikuju.

Kako je dataset veoma velik, empirijsko trazenje optimalnih parametara modela pomocu *GridSearch*-a nije bilo prakticno, pa su parametri izabrani na osnovu eksperimentalne analize.

4.2.4 Parametri modela

Korisceni su sledeci parametri modela logisticke regresije:

- `solver = saga`

  Odredjuje optimizacioni algoritam koji se koristi za treniranje modela. Izabran je **SAGA solver** jer je efikasan za rad sa velikim dataset-ovima i velikim brojem atributa. Takodje podrzava multinomial logisticku regresiju i paralelizaciju.

- `max_iter = 35`

  Predstavlja maksimalan broj iteracija koje optimizacioni algoritam moze izvrsiti tokom treniranja modela. Ova vrednost omogucava algoritmu dovoljno iteracija da konvergira ka stabilnom resenju bez nepotrebnog produzenja vremena treniranja.

- `tol = 1e-3`

  Definise toleranciju za kriterijum zaustavljanja optimizacije. Kada promena u vrednosti funkcije gubitka postane manja od ove vrednosti, algoritam smatra da je dostignuta konvergencija i prekida treniranje.

- `n_jobs = -1`

  Omogucava koriscenje svih dostupnih procesorskih jezgara tokom treniranja modela. Na taj nacin se odredjeni delovi optimizacije izvrsavaju paralelno, sto moze znacajno ubrzati proces treniranja.

- `verbose = 1`

  Ukljucuje ispis informacija o toku treniranja modela, sto omogucava pracenje napretka optimizacije i uvid u broj izvrsenih iteracija.

- `C = 1.0`

  Ako je C veliko, regularizacija je slaba. To znači da model nije mnogo kažnjen za velike koeficijente, pa pokušava da se što bolje uklopi u trening podatke. Granica odlučivanja može postati komplikovanija i model može da počne da hvata i šum u podacima, što povećava rizik od overfitting-a. Ako je C malo, regularizacija je jaka. Model tada snažno kažnjava velike koeficijente, pa je primoran da koristi manje vrednosti parametara i jednostavniju granicu odlučivanja. Takav model je obično stabilniji i bolje generalizuje, ali ako je C previše mali može doći do underfitting-a, jer model postaje previše ograničen.

- `random_state = 42`

  Postavlja inicijalno seme generatora slucajnih brojeva kako bi rezultati treniranja bili reproduktivni, odnosno kako bi isti eksperiment uvek davao iste rezultate.

Pre pokretanja treninga notebook proverava da li vec postoji **prethodno treniran model** sacuvan na disku. Ukoliko model postoji, on se ucitava i preskace se ponovno treniranje. Ovaj pristup omogucava da se eksperiment pokrene vise puta bez potrebe da se svaki put ponavlja dugotrajan proces treniranja, sto je bilo posebno korisno tokom testiranja i kasnije analize rezultata.

Nakon sto je model istreniran, koristi se za **predikciju nad test skupom podataka**. Model za svaki primer iz test skupa izracunava najverovatniju klasu i generise odgovarajucu predikciju. Dobijene predikcije se zatim porede sa stvarnim oznakama klasa kako bi se izracunale metrike performansi modela.

4.3 Analiza performansi i metrika

Nakon treniranja modela logisticke regresije, sledeci korak je procena njegovih performansi nad test skupom podataka. Cilj evaluacije je da se utvrdi koliko model uspesno klasifikuje karaktere koje nije video tokom treninga.
U ovom projektu za evaluaciju su koriscene tri standardne metrike klasifikacije:

- accuracy
- F1 score (macro)
- confusion matrix

Svaka od ovih metrika daje drugaciji uvid u ponasanje modela i zajedno omogucavaju detaljniju analizu rezultata.

4.3.1 Accuracy

Accuracy predstavlja odnos broja tacno klasifikovanih primera i ukupnog broja primera u test skupu. Jedna od mana ove metrike je nebalansiranost podataka. Na primer, ukoliko su podaci nebalansirani i model uvek predvidja najzastupljeniju klasu, moguce je dobiti veliku tacnost iako model zapravo nije naucio korisne obrasce.
U nasem slucaju, analizom dataset-a tokom faze preprocesiranja utvrdjeno je da su podaci relativno ravnomerno balansirani. Zbog toga accuracy metrika daje realnu sliku kvaliteta modela.
Dobijena vrednost iznosi

$$Accuracy = 0.477$$

sto znaci da model pravilno klasifikuje priblizno 47.7% karaktera iz test skupa. Drugim recima, od 100 karaktera koje model vidi, oko 48 ce biti tacno prepoznato, dok ce 52 biti pogresno klasifikovana.

Da bi se ova vrednost pravilno interpretirala, potrebno je uzeti u obzir broj klasa u problemu. U nasem slucaju model klasifikuje 256 razlicitih karaktera. Kada bi model nasumicno birao klasu, ocekivana tacnost bi bila

$$Accuracy_{random} = \frac{1}{256} \approx 0.0039$$

odnosno oko 0.39%. U poredjenju sa tim, rezultat od 47.7% je visestruko bolji od slucajnog pogadjanja i pokazuje da model uspeva da prepozna znacajan deo strukture u podacima.

Ipak, ova vrednost ukazuje da problem klasifikacije nije jednostavan. Logisticka regresija je linearan model koji pokusava da razdvoji klase linearnim granicama u prostoru atributa. Medjutim, vizuelni oblici karaktera cesto imaju slozene i nelinearne obrasce, narocito kada se uzmu u obzir razliciti fontovi, stilovi i debljina karaktera. Zbog toga model relativno uspesno prepoznaje neke karaktere, dok kod drugih dolazi do cestih konfuzija.

Zbog toga se ova vrednost accuracy moze posmatrati kao solidan pocetni rezultat (*baseline*) za ovakav problem sa velikim brojem klasa.

4.3.2 F1 score

F1 mera predstavlja kombinaciju preciznosti (*precision*) i odziva (*recall*). Visoka vrednost F1 mere postize se samo kada su obe ove mere visoke, cime se izbegava situacija da model deluje kvalitetno samo zbog jedne dobre metrike.

Preciznost meri koliko je model precizan u smislu koliko je stvarno tacnih klasa od svih za koje je model predvideo da su tacne. Sa druge strane, odziv predstavlja procenat stvarnih primera neke klase koje je model uspeo da pravilno prepozna.

U nasem projektu koristi se **macro F1 score**, koji racuna F1 vrednost za svaku klasu posebno, a zatim uzima njihov prosek. Dobijena vrednost iznosi

$$F1_{macro} = 0.336$$

Ova vrednost je primetno niza od accuracy rezultata (0.477), sto ukazuje da model ne postize jednako dobre performanse za sve karaktere. Postoje klase koje model prepoznaje relativno dobro, ali i klase kod kojih dolazi do velikog broja gresaka.

4.3.3 Weighted F1 score

Treca metrika koja je koristena je **weighted F1 score**. Ona se racuna na slican nacin kao macro F1, ali uz jednu vaznu razliku: svaka klasa dobija tezinu proporcionalnu broju svojih primera u dataset-u.

Dobijena vrednost iznosi

$$F1_{weighted} = 0.456$$

Ova vrednost je veoma bliska accuracy rezultatu, sto sugerise da model relativno dobro klasifikuje cesce karaktere u dataset-u, dok se najveci broj gresaka verovatno javlja kod redjih ili vizuelno kompleksnijih karaktera.

Razlika izmedju macro F1 i weighted F1 vrednosti daje vazan uvid u ponasanje modela. Posto je macro F1 znacajno nizi od weighted F1, to ukazuje da model ima vece poteskoce sa klasama koje se pojavljuju redje u dataset-u.

4.3.4 Matrica konfuzije

Pored metrika kao sto su accuracy i F1 score, za detaljniju analizu modela koriscena je i **matrica konfuzije** (*confusion matrix*). Ona prikazuje kako model klasifikuje svaku klasu i gde najcesce dolazi do gresaka.

Matrica konfuzije je kvadratna matrica dimenzija $N \times N$, gde je N broj klasa. U nasem slucaju problem obuhvata 256 razlicitih karaktera, pa matrica ima dimenzije 256×256. Redovi predstavljaju stvarne klase, dok kolone predstavljaju klase koje je model predvideo.
Elementi na glavnoj dijagonali predstavljaju tacno klasifikovane karaktere, dok elementi van dijagonale predstavljaju greske modela. Na osnovu ove matrice mozemo identifikovati koje karaktere model najcesce mesa.

Na osnovu konfuzione matrice izvedena je dodatna analiza koja ima za cilj da pronadje najcesce greske modela i njihovu vizuelizaciju. Rezultati ove analize sacuvani su u fajlu results/logistic_regression/top_confusions.csv.
Analiza pokazuje da model najcesce mesa karaktere koji su vizuelno slicni, kao sto su na primer:

- 0 i O
- 1 i l
- 4 i 9

Posebno je zanimljivo da se karakter `1` pojavljuje u velikom broju konfuzija. U mnogim fontovima ovaj karakter se prikazuje kao jednostavna vertikalna linija, koja moze podsecati na druge karaktere ili simbole, na primer znak `/`.
Ova analiza potvrdjuje da vecina gresaka modela ne nastaje nasumicno, vec je povezana sa vizuelnom slicnoscu izmedju odredjenih karaktera. To dodatno objasnjava zasto je klasifikacija ovog problema zahtevna, posebno kada se uzme u obzir veliki broj klasa i varijacije u fontovima i stilovima karaktera.

Napomena: svi rezultati analiza sacuvani su u posebnom direktorijumu `results`.

4.3.5 Per-class accuracy

Jos jedna zanimljiva analiza koja je sprovedena predstavlja **per-class accuracy**, odnosno analizu tacnosti po klasama. Za svaki karakter izracunat je odnos izmedju broja tacno klasifikovanih primera i ukupnog broja njegovih pojavljivanja u test skupu. Na ovaj nacin mozemo videti koje karaktere model prepoznaje najlakse, a koji predstavljaju najveci problem.

Rezultati pokazuju da postoje znacajne razlike izmedju pojedinih klasa. Najlosije klasifikovani karakteri imaju veoma nisku tacnost, u nekim slucajevima cak ispod 1%, sto znaci da model gotovo nikada ne uspeva da ih pravilno prepozna. Ovi karakteri su u dataset-u predstavljeni uglavnom kao prosireni ASCII simboli (npr. `\x99`, `\x97`, `\x9f`), koji se u razlicitim fontovima mogu prikazivati

veoma razlicito ili imati slozen vizuelni oblik. Zbog toga model tesko uspeva da pronadje stabilne obrasce koji bi ih jasno razlikovali od drugih klasa.

Sa druge strane, odredjeni karakteri postizu znatno vecu tacnost klasifikacije. Najbolje klasifikovani karakteri su uglavnom jednostavni numericki simboli, kao sto su `7`, `5`, `6`, `3` i `1`, gde tacnost dostize izmedju 70% i 84%. Ovi karakteri imaju relativno stabilan i prepoznatljiv oblik u vecini fontova, sto olaksava modelu da nauci njihove karakteristike.

Jos jedan zanimljiv zakljucak ove analize je da karakteri koji imaju veci broj primera u dataset-u cesto imaju i bolju tacnost klasifikacije. Na primer, karakteri kao sto su `1` i `0` pojavljuju se nekoliko hiljada puta u test skupu, sto modelu daje vise podataka na osnovu kojih moze da nauci njihove karakteristike.

4.3.6 Analiza uspesnosti modela po fontovima

Pored analize po klasama, izvrsena je i analiza uspesnosti modela u odnosu na font u kome je karakter generisan. Za svaki font izracunata je tacnost klasifikacije, odnosno odnos izmedju broja tacno klasifikovanih karaktera iz tog fonta i ukupnog broja karaktera koji pripadaju tom fontu u test skupu.

Rezultati pokazuju da performanse modela znacajno zavise od tipa fonta. Neki fontovi postizu veoma visoku tacnost klasifikacije. Na primer, font `NUMERICS` dostize tacnost od oko 96%, dok fontovi `E13B`, `OCRB` i `OCRA` takodje imaju veoma visoke vrednosti tacnosti (izmedju priblizno 88% i 96%). Ovi fontovi su specificni po tome sto imaju vrlo jednostavan i standardizovan oblik karaktera, cesto dizajniran upravo za masinsko citanje.

Sa druge strane, postoje fontovi kod kojih model postize veoma nisku tacnost klasifikacije. Na primer, fontovi kao sto su `TAHOMA`, `BUXTON`, `YI BAITI`, `SERIF` i `SKETCHFLOW` imaju tacnost manju od 10%. Ovi fontovi cesto imaju slozeniji stil karaktera, veci broj dekorativnih elemenata ili znacajne varijacije u obliku slova, sto otežava modelu da prepozna stabilne obrasce u podacima.

Ova analiza jasno pokazuje da je tip fonta jedan od kljucnih faktora koji utice na uspesnost klasifikacije. Logisticka regresija kao linearan model uspesno prepoznaje karaktere kada su njihovi oblici relativno jednostavni i konzistentni, ali ima znatno vise poteskoca sa fontovima koji uvode vecu vizuelnu kompleksnost ili stilizaciju karaktera.

4.3.7 Analiza uticaja italic stila i debljine karaktera

Pored analize po klasama i fontovima, posmatran je i uticaj stilskih karakteristika fonta, konkretno *italic* stil i debljina karaktera (*font weight*). Cilj ove analize bio je da se ispita kako promene u vizuelnom izgledu karaktera uticu na uspesnost klasifikacije.

Rezultati pokazuju jasnu razliku izmedju normalnih i italic karaktera. Za karaktere koji nisu italic model postize tacnost od oko 57.13%, dok za italic karaktere tacnost opada na priblizno 26.15%. Ova razlika ukazuje na to da italic stil znacajno otezava proces klasifikacije. Razlog je u tome sto italic font uvodi nagib karaktera, cime se menja raspored piksela i oblik slova u odnosu na standardni prikaz.

Slican efekat primecen je i u analizi debljine karaktera. Za karaktere cija se debljina nalazi u opsegu $0.4-0.6$ model postize tacnost od oko 56.9%, dok za deblje karaktere u opsegu $0.6-0.8$ tacnost znacajno opada na oko 27.4%.

Ovakav rezultat moze se objasniti promenom vizuelne strukture karaktera. Kako se debljina fonta povecava, pikseli koji cine karakter postaju siri i cesto se medjusobno spajaju, sto moze dovesti do toga da razliciti karakteri pocnu da lice jedni na druge. Na primer, kod debljih fontova moze doci do konfuzije izmedju karaktera kao sto su `8` i `B`, `5` i `S`, ili `O` i `0`, jer dodatna debljina linija smanjuje vizuelne razlike izmedju njihovih oblika.

4.4 PCA redukcija

Pre nego sto analiziramo konkretne rezultate, potrebno je formulisati ocekivanje o tome kako ce primena PCA uticati na model logisticke regresije. Logisticka regresija je linearan model koji uci tezine za svaki pojedinacni atribut u prostoru osobina. U originalnom prostoru podataka model optimizuje linearni izraz koji ukljucuje sve ulazne atribute, pri cemu medju njima moze postojati znacajna medjusobna korelacija. PCA transformacijom originalni skup atributa projektuje se u novi skup ortogonalnih komponenti koje nisu medjusobno korelisane. Time se smanjuje dimenzionalnost prostora i broj parametara koje model mora da nauci. Zbog toga se u praksi cesto ocekuje da PCA moze dovesti do stabilnije optimizacije i brzeg treninga modela, dok se performanse modela najcesce ne menjaju znacajno ili se blago poboljsavaju.

Model	Accuracy	Macro F1	Weighted F1
Logistic Regression	0.47768	0.336350	0.456242
Logistic Regression + PCA	0.46265	0.313373	0.436223

Table 1: Poredjenje performansi logisticke regresije sa i bez primene PCA.

Medjutim, rezultati dobijeni u eksperimentu pokazuju da primena PCA u ovom slucaju nije dovela do poboljsanja performansi modela. Bez primene PCA logisticka regresija ostvaruje tacnost od 0.47768, makro F1 meru 0.336350 i ponderisanu F1 meru 0.456242. Nakon primene PCA, tacnost modela opada na 0.46265, makro F1 mera na 0.313373, dok ponderisana F1 mera iznosi 0.436223. Moze se primetiti da su sve metrike blago smanjene, sto ukazuje na to da redukcija dimenzionalnosti u ovom slucaju uklanja deo (mali, ali merljiv) informacija koje su bile korisne za razdvajanje klasa.

Sa druge strane, znacajna prednost PCA u ovom eksperimentu ogleda se u vremenu treninga modela. Trening logisticke regresije nad originalnim podacima trajao je priblizno 7540 sekundi (2h), dok je nakon redukcije dimenzionalnosti pomocu PCA vreme treninga smanjeno na oko 855.8 sekundi (15ak minuta). Ovo predstavlja ubrzanje od gotovo devet puta. Razlog za ovako znacajno ubrzanje lezi u cinjenici da je broj atributa smanjen sa 400 na 119, cime se smanjuje broj parametara koje optimizacioni algoritam mora da proceni.

Na osnovu ovih rezultata moze se zakljuciti da PCA u ovom konkretnom problemu nije doprineo poboljsanju klasifikacionih performansi logisticke regresije, ali je znacajno smanjio racunarsku slozenost i vreme potrebno za trening modela. Takav rezultat je u skladu sa ocekivanjem da PCA prvenstveno sluzi kao tehnika za redukciju dimenzionalnosti i ubrzavanje rada modela, dok poboljsanje tacnosti nije zagarantovano ukoliko se tokom projekcije izgubi deo diskriminativnih informacija iz originalnog prostora atributa.

5 Support Vector Machine (SVM)

5.1 Koncept

Metod potpornih vektora (Support Vector Machine – SVM) predstavlja nadgledani model masinskog ucenja koji se zasniva na geometrijskoj interpretaciji podataka u vektorskom prostoru. Osnovna ideja ovog pristupa je pronalazenje razdvajajuce hiperravni koja deli podatke razlicitih klasa tako da se maksimizuje margina, odnosno rastojanje izmedju hiperravni i najblizih tacaka iz svake klase. Na taj nacin model ne trazi bilo koju granicu razdvajanja, vec onu koja obezbedjuje najvecu mogucu robusnost klasifikacije. Tacke koje se nalaze najblize granici razdvajanja nazivaju se *potporni vektori* i one imaju kljucni uticaj na polozaj hiperravni, dok vecina ostalih trening primera nema direktan uticaj na konacno resenje modela.

U svojoj osnovnoj formi SVM resava binarni klasifikacioni problem u kome je svaka instanca predstavljena vektorom atributa x_i, dok je oznaka klase $y_i \in \{-1, +1\}$. Cilj algoritma je pronalazenje parametara w i b koji definisu hiperravan

$$w \cdot x + b = 0$$

Klasifikacija nove instance vrsi se na osnovu znaka izraza $w \cdot x + b$, pri cemu apsolutna vrednost ovog izraza predstavlja udaljenost instance od razdvajajuce ravni. Problem pronalazenja optimalne hiperravni moze se formulisati kao optimizacioni problem u kome se minimizuje norma vektora tezina uz ogranicenje da sve trening instance budu pravilno klasifikovane i da se nalaze izvan margine. U praksi se najcesce koristi tzv. *soft-margin* formulacija, gde se uvodi regularizacioni parametar koji omogucava odredjeni broj gresaka u klasifikaciji kako bi se dobio stabilniji model na realnim podacima.

U ovom projektu koriscena je linearna varijanta SVM modela implementirana kroz algoritam `LinearSVC`. Posto posmatrani problem podrazumeva klasifikaciju 256 razlicitih ASCII karaktera, primenjena je strategija *one-vs-rest*, gde se za svaku klasu trenira poseban binarni klasifikator koji razlikuje jednu klasu od svih ostalih. Svaki od ovih klasifikatora uci sopstveni vektor tezina koji definise hiperravan u prostoru atributa. U nasem slucaju atributi predstavljaju piksele slike karaktera dimenzije 20×20, pa model zapravo pronalazi linearne granice razdvajanja u prostoru od 400 dimenzija. Na osnovu vrednosti funkcije odlucivanja za svaku klasu, model zatim bira onu klasu za koju je vrednost najveca, cime se dobija konacna predikcija karaktera. Ovakav pristup je posebno pogodan za probleme sa velikim brojem atributa i omogucava efikasno treniranje cak i na velikim skupovima podataka.

5.2 Konstrukcija modela

Kao i u prethodnom eksperimentu, model je implementiran koriscenjem `Pipeline` objekta iz biblioteke *scikit-learn*. Prvi korak pipeline-a predstavlja standardizacija atributa pomocu transformacije `StandardScaler`. Standardizacija je posebno vazna kod SVM algoritma jer se model zasniva na skalarnim proizvodima izmedju vektora atributa prilikom odredjivanja razdvajajuce hiperravni. Ukoliko atributi imaju razlicite skale, pojedine karakteristike mogu dominirati u procesu optimizacije. Standardizacija obezbedjuje da svi atributi imaju uporediv uticaj na formiranje modela.

Nakon standardizacije primenjuje se klasifikator `LinearSVC`, koji predstavlja linearnu implementaciju algoritma potpornih vektora. Za razliku od klasicne formulacije SVM-a koja resava dualni optimizacioni problem i eksplicitno identifikuje potporne vektore, implementacija `LinearSVC` koristi optimizacioni algoritam iz biblioteke *LIBLINEAR*. Ovaj algoritam rešava optimizacioni problem i posebno je optimizovan za rad sa velikim dataset-ovima i velikim brojem atributa.

U kontekstu ovog projekta svaki uzorak predstavlja sliku karaktera dimenzije 20×20 piksela, što odgovara vektoru od 400 atributa. Linearni SVM model stoga pronalazi razdvajajuće hiperravni u prostoru od 400 dimenzija koje omogućavaju razdvajanje različitih klasa karaktera.

Pošto dataset sadrži ukupno 256 mogućih ASCII karaktera, problem klasifikacije je multiclass prirode. U implementaciji se koristi strategija *one-vs-rest*, gde se za svaku klasu trenira poseban

binarni klasifikator koji razlikuje jednu klasu od svih ostalih. Konacna klasifikacija nove instance dobija se poredjenjem vrednosti funkcije odlucivanja svih klasifikatora i izborom klase sa najvecom vrednošću.

S obzirom na veličinu dataset-a, iscrpno pretrazivanje prostora parametara pomoću metoda kao što je *GridSearch* nije bilo praktično. Parametri modela su zato odredkeni na osnovu eksperimentalne analize i preporuka iz dokumentacije biblioteke *scikit-learn*.

Korisceni su sledeci parametri modela linearnog SVM klasifikatora:

- `C = 1.0`

  Predstavlja parametar regularizacije koji kontrolise kompromis izmedju maksimizacije margine i dozvoljene klasifikacione greske na trening podacima. Vece vrednosti parametra `C` smanjuju regularizaciju i omogucavaju modelu da se bolje prilagodi trening podacima, dok manje vrednosti uvode jaca ogranicenja nad parametrima modela i mogu doprineti boljoj generalizaciji na nevidjenim podacima.

- `max_iter = 1000`

  Definise maksimalan broj iteracija optimizacionog algoritma tokom treniranja modela. Ovaj parametar ogranicava ukupno vreme optimizacije i obezbedjuje da proces treniranja ne traje neograniceno dugo. U praksi se algoritam najcesce zaustavlja ranije ukoliko je dostignuta konvergencija.

- `tol = 1e-2`

  Predstavlja toleranciju za kriterijum zaustavljanja optimizacije. Kada promena u vrednosti funkcije cilja postane manja od zadate tolerancije, algoritam smatra da je dostignuta konvergencija i prekida dalji proces treniranja.

- `dual = False`

  Ovaj parametar odredjuje nacin na koji se resava optimizacioni problem tokom treniranja SVM modela. U praksi postoje dva pristupa, ali izbor izmedju njih zavisi od odnosa izmedju broja instanci i broja atributa u skupu podataka. Kada je broj trening instanci znatno veci od broja atributa, efikasnije je resavati problem direktno u prostoru atributa. U nasem slucaju svaka instanca je opisana sa 400 atributa (pikseli slike), dok skup podataka sadrzi stotine hiljada instanci. Zbog toga je ovakav pristup racunski jednostavniji i dovodi do brzeg treniranja modela.

- `multi_class = "ovr"`

  Odredjuje strategiju za resavanje problema klasifikacije sa vise klasa. Koristi se pristup *one-vs-rest*, gde se za svaku klasu trenira poseban binarni klasifikator koji razlikuje jednu klasu od svih ostalih. Konačna predikcija dobija se izborom klase sa najvećom vrednošću funkcije odlučivanja.

- `loss = "squared_hinge"`

  Definise funkciju gubitka koja se koristi tokom treniranja modela. Squared hinge loss predstavlja modifikaciju standardne hinge funkcije gubitka koja uvodi kvadratnu penalizaciju za pogresno klasifikovane primere. Ova varijanta cesto dovodi do stabilnijeg optimizacionog procesa i brze konvergencije.

- `verbose = 1`

  Ukljucuje ispis informacija o toku treniranja modela, sto omogucava pracenje napretka optimizacije i uvid u broj izvrsenih iteracija.

- `random_state = 42`

  Postavlja inicijalno seme generatora slucajnih brojeva kako bi rezultati treniranja bili reproduktivni, odnosno kako bi isti eksperiment uvek davao iste rezultate.

U odnosu na prethodno korisceni model logističke regresije, treniranje linearnog SVM klasifikatora zahtevalo je znatno vise vremena. Glavni razlog za to je veličina dataset-a i način na koji se resava problem klasifikacije sa velikim brojem klasa.

Dataset koji se koristi u ovom radu sadrzi vise stotina hiljada instanci, pri cemu je svaka instanca predstavljena vektorom od 400 atributa koji odgovaraju pikselima slike karaktera. Tokom treniranja SVM model mora da resava optimizacioni problem visoke dimenzionalnosti, gde se iterativno azuriraju parametri modela sve dok se ne dostigne konvergencija.

Dodatni faktor koji utice na vreme treniranja jeste cinjenica da problem klasifikacije obuhvata ukupno 256 klasa. Posto se koristi *one-vs-rest* strategija, za svaku klasu trenira se poseban binarni klasifikator koji razlikuje jednu klasu od svih ostalih. To znaci da se optimizacioni postupak mora izvrsiti ukupno 256 puta, odnosno za svaki ASCII karakter posebno.

Kombinacija velikog broja instanci, relativno visokog broja atributa i velikog broja klasa dovodi do znacajnog povecanja ukupnog vremena treniranja modela. Iako implementacija `LinearSVC` koristi optimizovani algoritam iz biblioteke *LIBLINEAR*, koji je dizajniran za rad sa velikim datasetovima, ukupno vreme treniranja moze biti znacajno kada se model trenira nad kompletnim skupom podataka i velikim brojem klasa.

5.3 Rezultati modela

Evaluacija modela nad test skupom pokazuje da linearni SVM postize tacnost od 0.4102, uz vrednosti 0.2266 za macro F1 i 0.3522 za weighted F1 meru.

Iako tacnost od oko 41% na prvi pogled moze delovati umereno, treba imati u vidu da zadatak obuhvata klasifikaciju cak 256 razlicitih klasa. Model mora da razlikuje veliki broj vizuelno slicnih karaktera koji se pojavljuju u razlicitim fontovima, debljinama linije i stilovima ispisa, sto znacajno otezava problem.

Razlika izmedju macro i weighted F1 mere ukazuje na neujednacene performanse po klasama. Niza vrednost macro F1 (0.2266) sugerise da model ima slabije rezultate za deo klasa, dok veca vrednost weighted F1 (0.3522) pokazuje da model bolje klasifikuje klase koje su zastupljenije u dataset-u.

Analiza najcescih konfuzija izmedju klasa pokazuje da se greske modela najcesce javljaju izmedju vizuelno veoma slicnih karaktera. Na primer, cifra `1` se cesto klasifikuje kao znak `/`, dok se slovo `O` cesto prepoznaje kao cifra `0`. Slican obrazac moze se uociti i kod parova kao sto su `4` i `9`, `I` i `1`, kao i `8` i `6`. Ovakve greske su ocekivane jer se mnogi ASCII karakteri u razlicitim fontovima mogu pojaviti sa vrlo slicnim vizuelnim oblicima. Kada se uzme u obzir da dataset sadrzi veliki broj fontova, stilova i debljina linija, pojedini karakteri mogu imati veoma slicnu raspodelu piksela u prostoru atributa, sto otezava njihovo razdvajanje linearnim modelom. Ovi rezultati potvrdjuju da najveci deo klasifikacionih gresaka ne potice od potpuno pogresnog prepoznavanja karaktera, vec od konfuzije izmedju graficki slicnih simbola.

Analiza tacnosti po pojedinacnim klasama pokazuje znacajne razlike u performansama modela izmedju razlicitih karaktera. Najbolje klasifikovani karakteri uglavnom pripadaju grupi jednostavnijih i vizuelno jasno definisanih simbola, kao sto su cifre (`0`, `3`, `5`, `6`, `7`) ili znakovi sa karakteristicnim oblikom. Tacnost za ove karaktere dostize vrednosti izmedju približno 72% i 84%, sto ukazuje da model relativno uspesno prepoznaje njihove stabilne vizuelne obrasce.

Sa druge strane, najlosije klasifikovane klase gotovo u potpunosti pripadaju nestampajucim (*non-printable*) ASCII karakterima. Ovakva raspodela rezultata dodatno objasnjava nizu vrednost macro F1 mere, posto ona daje jednaku tezinu svim klasama, ukljucujuci i one za koje model ne uspeva da nauci pouzdane obrasce klasifikacije. Ovi rezultati takodje otvaraju pitanje da li specijalni ASCII karakteri uopste treba da budu ukljuceni u trening skup.

Analiza tacnosti po razlicitim fontovima pokazuje da performanse modela znacajno zavise od tipografskog stila u kome su karakteri generisani. Za pojedine fontove model postize veoma visoku tacnost, kao sto su `NUMERICS` i `E13B`, gde tacnost prelazi 95%, dok su fontovi `OCRB` i `OCRA` takodje dobro prepoznati sa tacnoscu iznad 80%. Ovakvi rezultati ukazuju da model lakse prepoznaje karaktere koji imaju jasno definisane i standardizovane oblike sa malom varijacijom izmedju

instanci.

Sa druge strane, postoje fontovi za koje model ostvaruje izrazito slabe rezultate. Na primer, fontovi `CONSOLAS`, `BUXTON`, `SKETCHFLOW`, `SERIF` i `TAHOMA` imaju tacnost manju od 10%. Ovi fontovi cesto sadrze stilizovane ili kompleksnije oblike karaktera sa razlicitim zakrivljenjima, debljinama linija ili dekorativnim elementima, sto znacajno menja raspodelu piksela u prostoru atributa i otezuje linearnom modelu razdvajanje klasa.

Ova analiza pokazuje da tipografske razlike predstavljaju jedan od vaznih faktora koji uticu na performanse klasifikacije. Linearni SVM model dobro radi kada je oblik karaktera konzistentan, dok veca varijabilnost oblika kod pojedinih fontova dovodi do znacajnog pada tacnosti klasifikacije.

Dodatna analiza uticaja stilskih varijacija karaktera pokazuje da promene u nacinu ispisa znacajno uticu na performanse modela. Kada se posmatra stil ispisa, model postize tacnost od oko 51% nad regularnim karakterima, dok tacnost nad italic varijantama opada na priblizno 18%.

Slican obrazac uocava se i pri analizi debljine linije (*strength*). Karakteri sa umerenim vrednostima debljine (0.4–0.6) ostvaruju tacnost od oko 51%, dok kod debljih karaktera (0.6–0.8) tacnost opada na priblizno 18%.

Ovi rezultati ukazuju da model pokazuje relativno ograničenu robustnost na stilističke transformacije karaktera. Promene kao što su nagib ili debljina linije mogu značajno promeniti vizuelni obrazac karaktera, što dovodi do smanjenja tačnosti klasifikacije.

5.4 Vizualizacija diskriminativnih piksela

Kako bi se bolje razumela struktura podataka i nacin na koji model razlikuje pojedine karaktere, izvrsena je vizualizacija *difference-of-means* reprezentacije za odabrane klase. Ova metoda omogucava identifikaciju piksela koji su karakteristicni za odredjeni karakter u odnosu na sve ostale karaktere u skupu podataka.

Za svaku klasu c racuna se prosecna slika svih instanci tog karaktera, kao i prosecna slika svih ostalih karaktera u datasetu. Formalno, neka je x_i vektor piksela slike, a y_i oznaka klase. Prosecna slika klase definiše se kao

$$\mu_c = \frac{1}{N_c} \sum_{i:y_i=c} x_i$$

dok je prosečna slika svih ostalih klasa

$$\mu_{rest} = \frac{1}{N - N_c} \sum_{i:y_i \neq c} x_i.$$

Vizualizacija se dobija kao razlika ove dve vrednosti:

$$D_c = \mu_c - \mu_{rest}.$$

Dobijeni vektor dimenzije 400 zatim se preslikava u matricu dimenzije 20×20, cime se dobija slika koja prikazuje piksele koji su najkarakteristicniji za datu klasu.

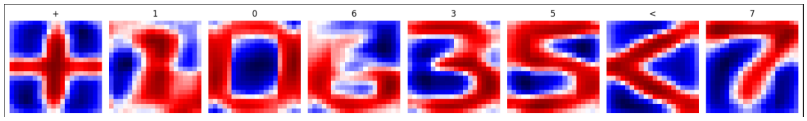


Figure 3: SVM difference of means

U ovim vizualizacijama crvene vrednosti predstavljaju piksele koji su cesce aktivni kod posmatrane klase nego kod ostalih karaktera, dok plave vrednosti predstavljaju piksele koji su tipicniji za ostale karaktere. Na taj nacin moguce je intuitivno uociti koji delovi slike nose najvise diskriminativne informacije za klasifikaciju.

Ova analiza pokazuje da linearni klasifikator u velikoj meri zasniva svoju odluku na prostornom rasporedu karakteristicnih piksela. Karakteri koji imaju stabilan i jedinstven vizuelni oblik u različitim fontovima lakše se razdvajaju u prostoru atributa, dok karakteri sa vecim varijacijama oblika ili slicnostima sa drugim simbolima predstavljaju znatno tezi problem klasifikacije.

Napomena: Za rezultate pogledati notebook src/models/svm.ipynb ili rezultat svake analize posebno u results/svm/analiza.csv

5.5 PCA redukcija

SVM predstavlja linearan klasifikator koji pronalazi hiperravan koja maksimalno razdvaja klase u prostoru osobina. Kod linearne varijante SVM-a, performanse modela mogu zavisiti od dimenzionalnosti prostora i strukture ulaznih atributa. U nekim slucajevima primena PCA moze doprineti poboljsanju performansi jer uklanja redundansu i korelaciju medju atributima, projektujuci podatke u ortogonalan prostor manjih dimenzija. Time se ponekad dobija stabilniji model i smanjuje racunarska slozenost treninga.

Sa druge strane, SVM modeli cesto dobro funkcionisu i u prostorima visoke dimenzionalnosti. Kada se primeni PCA, deo originalnih atributa se odbacuje, pri cemu komponente sa manjom varijansom mogu sadrzati informacije koje su znacajne za razdvajanje pojedinih klasa. U takvim situacijama redukcija dimenzionalnosti moze dovesti do gubitka diskriminativnih informacija i blagog pogorsanja performansi modela.

Model	Accuracy	Macro F1	Weighted F1
SVM	0.410235	0.226577	0.352163
SVM + PCA	0.386161	0.184060	0.317281

Table 2: Poredjenje performansi SVM modela sa i bez primene PCA.

Rezultati dobijeni u eksperimentu pokazuju upravo takvo ponasanje. SVM model treniran nad originalnim preprocesiranim podacima ostvaruje tacnost od 0.410235, makro F1 meru 0.226577 i ponderisanu F1 meru 0.352163. Nakon primene PCA dolazi do pada svih metrika: tacnost opada na 0.386161, makro F1 mera na 0.184060, dok ponderisana F1 mera iznosi 0.317281. Ovakav rezultat ukazuje na to da je redukcija dimenzionalnosti uklonila deo informacija koje su bile korisne za razdvajanje klasa u prostoru osobina.

Ipak, kao i kod logisticke regresije, primena PCA ima znacajan uticaj na vreme treninga modela. Trening SVM modela nad originalnim podacima trajao je oko 13744 sekunde, sto odgovara vremenu od priblizno 3 sata i 49 minuta. Nakon redukcije dimenzionalnosti pomocu PCA vreme treninga smanjeno je na oko 3718 sekundi, odnosno priblizno 1 sat i 2 minuta. Ovo predstavlja znacajno ubrzanje treninga modela. Razlog za takvo smanjenje vremena lezi u smanjenju dimenzionalnosti prostora osobina sa 400 na 119, cime se smanjuje broj parametara koje optimizacioni algoritam mora da proceni.

Na osnovu ovih rezultata moze se zakljuciti da primena PCA u ovom slucaju nije doprinela poboljsanju klasifikacionih performansi SVM modela, ali je znacajno smanjila vreme potrebno za trening modela. Ovakav rezultat ukazuje na kompromis izmedju racunarske efikasnosti i klasifikacionih performansi, gde PCA doprinosi ubrzanju procesa ucenja, ali moze dovesti do gubitka dela informacija iz originalnog prostora atributa.

6 Random Forest

6.1 Koncept

Random Forest predstavlja ansambl metod klasifikacije zasnovan na skupu odlucionih stabala. Ideja algoritma je da se umesto jednog modela trenira veci broj stabala, pri cemu svako stablo uci na malo drugacijem skupu podataka i sa razlicitim podskupom atributa. Konacna odluka modela dobija se agregacijom predikcija svih stabala, najcesce glasanjem (*majority voting*). Ovakav pristup znacajno smanjuje varijansu modela i smanjuje rizik od preprilagodjavanja (*overfitting*), koji je karakteristican za pojedinacna odluciona stabla.

Osnovni mehanizam koji omogucava raznolikost stabala u Random Forest-u zasniva se na dve tehnike. Prva je *bootstrap* uzorkovanje (*bagging*), gde se za svako stablo nasumicno bira podskup trening podataka sa vracanjem. Na taj nacin svako stablo vidi malo drugaciju verziju skupa podataka. Druga tehnika odnosi se na izbor atributa prilikom svakog razdvajanja u stablu. Umesto da se razmatraju svi atributi, algoritam razmatra samo nasumicni podskup atributa, cime se dodatno povecava raznolikost stabala i smanjuje korelacija izmedju njih. Kombinacijom ovih mehanizama dobija se stabilan i robustan model koji dobro generalizuje na novim podacima.

U okviru ovog projekta za implementaciju algoritma koriscena je klasa *RandomForestClassifier* iz biblioteke *scikit-learn*. Ova implementacija omogucava efikasno treniranje velikog broja stabala i paralelizaciju procesa treniranja. Model je konfigurisan tako da koristi vise stabala srednje dubine, cime se postize balans izmedju slozenosti modela i vremena treniranja. Broj stabala kontrolisan je parametrom `n_estimators`, dok je maksimalna dubina stabla ogranicena parametrom `max_depth` kako bi se sprecilo prekomerno grananje i smanjilo vreme treniranja. Parametar `max_features` definise broj atributa koji se razmatraju prilikom svakog razdvajanja u stablu, sto predstavlja kljucni mehanizam za uvodjenje slucajnosti u model.

Zahvaljujuci ovakvoj konfiguraciji, model koristi vise relativno jednostavnih stabala koja zajedno formiraju mocan klasifikator sposoban da modeluje kompleksne nelinearne odnose u podacima. Random Forest se posebno dobro pokazuje na problemima sa velikim brojem atributa i velikim skupovima podataka, sto ga cini pogodnim izborom za dataset koji se koristi u ovom projektu.

6.2 Konstrukcija modela

Za implementaciju Random Forest modela koriscena je klasa *RandomForestClassifier* iz biblioteke *scikit-learn*. Prilikom konstrukcije modela bilo je potrebno odabrati skup hiperparametara koji obezbedjuju dobar balans izmedju tacnosti modela, vremena treniranja i potrosnje memorije, posebno imajuci u vidu velicinu skupa podataka (667k uzoraka, 400 atributa i 256 klasa).

U opstem slucaju izbor parametara se vrsi suzavanjem njihovog domena i pokretanjem *GridSearch* mehanizma. Medjutim, zbog izuzetno velikog skupa podataka ovakav pristup je u praksi neizvodljiv, pa su parametri odabrani empirijski kroz niz eksperimenata.

Parametar `n_estimators` Broj stabala u sumi kontrolise parametar `n_estimators`. U pocetnom pokusaju izabrana je vrednost `120`, ali je zbog velicine dataset-a RAM memorija brzo popunjena (16GB), nakon cega je sistem poceo da koristi spor SWAP mehanizam i trening je trajao preko 6 sati. Problem je resen restartovanjem sistema i smanjenjem broja stabala na `n_estimators = 40`. Ovaj model je istreniran za svega nekoliko minuta uz iznenadjujuce dobre rezultate. Pokusaj povecanja na `60` stabala nije doneo primetno poboljsanje tacnosti, pa je zakljuceno da model prakticno konvergira vec sa 40 stabala.

Parametar `max_depth` Parametar `max_depth` ogranicava maksimalnu dubinu svakog stabla. U ovom radu izabrana je vrednost `20`, cime se sprecava nekontrolisano grananje stabla koje bi moglo dovesti do preprilagodjavanja i znacajnog povecanja vremena treniranja.

Bez ovog ogranicenja stabla mogu postati veoma duboka, sto znacajno povecava broj cvorova i memorijske zahteve modela. Ogranicavanje dubine predstavlja kompromis izmedju ekspresivne moci modela i njegove sposobnosti generalizacije.

Parametar `max_features` Parametar `max_features` definise koliko atributa se razmatra prilikom svakog razdvajanja u stablu. U ovom modelu koristi se opcija `"sqrt"`, sto znaci da se pri svakom splitu razmatra koren ukupnog broja atributa.

Ova strategija je standardna preporuka za Random Forest klasifikatore jer povecava raznolikost stabala i smanjuje korelaciju izmedju njih, a istovremeno ubrzava trening.

Parametri `min_samples_split` i `min_samples_leaf` Ovi parametri kontrolisu minimalan broj uzoraka potreban za dalje grananje stabla i formiranje listova. U ovom radu izabrane su vrednosti `10` i `5`. S obzirom na veliki broj uzoraka u dataset-u, ovakav izbor doprinosi stabilnijem treningu i kracem vremenu treniranja modela.

Bootstrap uzorkovanje Parametar `bootstrap` postavljen je na `True`, cime se omogucava bootstrap uzorkovanje trening podataka prilikom treniranja svakog stabla. Ovaj mehanizam predstavlja kljucni deo Random Forest algoritma jer obezbedjuje razlicit trening skup za svako stablo, cime se povecava raznolikost modela i poboljsava generalizacija.

Paralelizacija treniranja Zbog velike velicine dataset-a posebna paznja posvecena je paralelizaciji treninga. Parametar `n_jobs` postavljen je na vrednost `3`, sto omogucava paralelno treniranje vise stabala uz kontrolu potrosnje memorije.

U praksi je primeceno da paralelizacija na svih 8 jezgara znacajno povecava potrosnju RAM memorije i dovodi do pada procesa. Nakon vise eksperimenata i pracenja zauzeca memorije, vrednost `3` se pokazala kao stabilno resenje.

Reproduktivnost eksperimenta Parametar `random_state` postavljen je na vrednost `42` kako bi se obezbedila reproduktivnost eksperimenta i mogucnost ponovnog dobijanja istih rezultata.

Cuvanje modela Tokom razvoja sistema implementirana je mogucnost cuvanja i ponovnog ucitavanja istreniranog modela pomocu `.pkl` fajla kako bi se izbeglo ponovno treniranje modela. Medjutim, zbog strukture Random Forest algoritma serijalizovani model zauzima izuzetno veliku kolicinu memorije. Svako stablo sadrzi veliki broj cvorova koji cuvaju informacije o atributima, pragovima razdvajanja i pokazivacima na potomke. Zbog velicine dataset-a stabla postaju veoma razgranata, pa je ukupna velicina modela dostigla priblizno 9 GB.

Tako velika velicina modela znacajno usporava ucitavanje i predstavlja problem prilikom distribucije projekta. Zbog toga je u konacnoj verziji implementacije uklonjena opcija ucitavanja modela, a model se trenira direktno prilikom pokretanja eksperimenta (oko 3 minuta na mom racunaru).

6.3 Rezultati modela

Nakon treniranja Random Forest modela, izvršena je evaluacija performansi na test skupu podataka korišćenjem standardnih metrika za klasifikaciju: tačnosti (accuracy), makro F1 mere (macro F1) i ponderisane F1 mere (weighted F1). Dobijeni rezultati su sledeći:

- Accuracy: 0.7081
- Macro F1: 0.6004
- Weighted F1: 0.6978

Dobijeni rezultati ukazuju na vrlo dobre performanse modela s obzirom na složenost samog problema. Dataset koji se koristi u ovom projektu sadrži veliki broj različitih klasa (256 karaktera), pri čemu mnoge klase imaju veoma sličan vizuelni izgled. Pored toga, karakteri potiču iz velikog broja različitih fontova, stilova i varijacija prikaza, što dodatno otežava proces klasifikacije.

U takvom okruženju postizanje tačnosti od približno 71% predstavlja veoma dobar rezultat. Model je uspeo da nauči kompleksne nelinearne odnose izmeu atributa koji predstavljaju piksele slike i odgovarajućih klasa karaktera. Random Forest algoritam je posebno pogodan za ovakav tip problema jer kombinuje veliki broj stabala odlucivanja, čime se postiže robustan model koji je sposoban da identifikuje različite obrasce u podacima.

Makro F1 mera od oko 0.60 ukazuje na to da model postiže solidne performanse kroz veliki broj različitih klasa, uključujući i one koje su slabije zastupljene u datasetu. Sa druge strane, ponderisana F1 mera bliska vrednosti tačnosti pokazuje da model veoma dobro klasifikuje i dominantnije klase u skupu podataka.

Sveukupno posmatrano, Random Forest model pokazuje sposobnost da uspešno generalizuje na velikom i heterogenom skupu podataka, postizuci stabilne i pouzdane rezultate u klasifikaciji karaktera.

Pored opštih metrika klasifikacije, izvršena je i analiza najčešćih grešaka koje model pravi. U tu svrhu identifikovani su parovi karaktera koji se najčešće medjusobno mešaju, odnosno slučajevi u kojima model predvidja jednu klasu umesto druge.

Rezultati pokazuju da se najveći broj konfuzija javlja izmedju karaktera koji imaju veoma sličan vizuelni oblik. Na primer, karakter `O` se relativno često klasifikuje kao cifra `0`. Vidimo da model pogresno klasifikuje cifre 1,2 i 3 sa njihovim verzijama kao eksponentima sto pokazuje koliko model dobro "hvata" slicnosti izmedju 2 oblika cak i kada se oni ne podudaraju po svojim piksel pozicijama.

Iz slicnih razloga se pojedini simboli poput crtice ili donje crte često meusobno mešaju. Razlog za to je činjenica da su ovi karakteri predstavljeni veoma jednostavnom geometrijom, sa malim brojem aktivnih piksela, što otežava jasno razdvajanje izmedju klasa. Svakako, mozemo biti vrlo zadovoljni ovim rezultatom.

U daljoj analizi najbolje i najlosije klasifikovanih karaktera, vidimo da model (kao i oni pre) ima najveci problem sa ASCII kodovima koji cak ni nemaju vizuelnu reprezentaciju u terminalu. Medjutim, on uspeva da i u tim, dodatno komplikovanim situacijama, klasifikuje karakter sa tacnoscu od 20 posto. U ovim okolnostima, mozemo smatrati ovu uspesnost jako dobrom. Sa druge strane, najbolje klasifikovani karakteri su upravo sve cifre 0-9, model ih je toliko dobro naucio da vrlo retko gresi, svi procenti uspesnosti su veci od 93, fenomenalan rezultat.

Kada je analiza fonta u pitanju, vidimo takodje slicno dobre rezultate. Kod onih koji su generalno laksi (posmatrajuci rezultate i prethodnih modela), random forest ima neverovatnu tacnost od 98+ procenata, sto govori da je skoro savrsen. Sa druge strane, kada su komplikovaniji fontovi u pitanju (za ovaj model to su GADUGI, GOTHICE, SCRIPT..) model ima uspesnost od 20 procenata u proseku. Ovo takodje mozemo posmatrati kao solidan rezultat.

Zavrsna analiza podrazumeva tacnost nad italic stilom karaktera, ocekivano model bolje klasifikuje one koji nisu iskoseni (cak 78 posto uspesnosti), ali daleko je od loseg na tezoj varijanti gde su fontovi iskoseni (53 procenta uspesnosti). I ova analiza pokazuje da je model vrlo dobar.

Slicne rezultate model postize i na analizi uspesnosti prema vrednostima strength atributa tj debljini samih karaktera. Ponovo, mozemo biti i vise nego zadovoljni rezultatom na debljim karakterima (oko 54 procenata uspesnosti).

6.4 Analiza važnosti piksela

Jedna od prednosti Random Forest modela jeste mogućnost procene značaja pojedinačnih atributa. U ovom slučaju svaki atribut predstavlja jedan piksel slike karaktera, pa se dobijene vrednosti važnosti mogu interpretirati kao procena doprinosa pojedinačnih piksela procesu klasifikacije. Na osnovu atributa `feature_importances_` iz treniranog modela konstruisana je mapa važnosti piksela dimenzije 20×20, koja odgovara prostornom rasporedu piksela u ulaznim slikama karaktera.

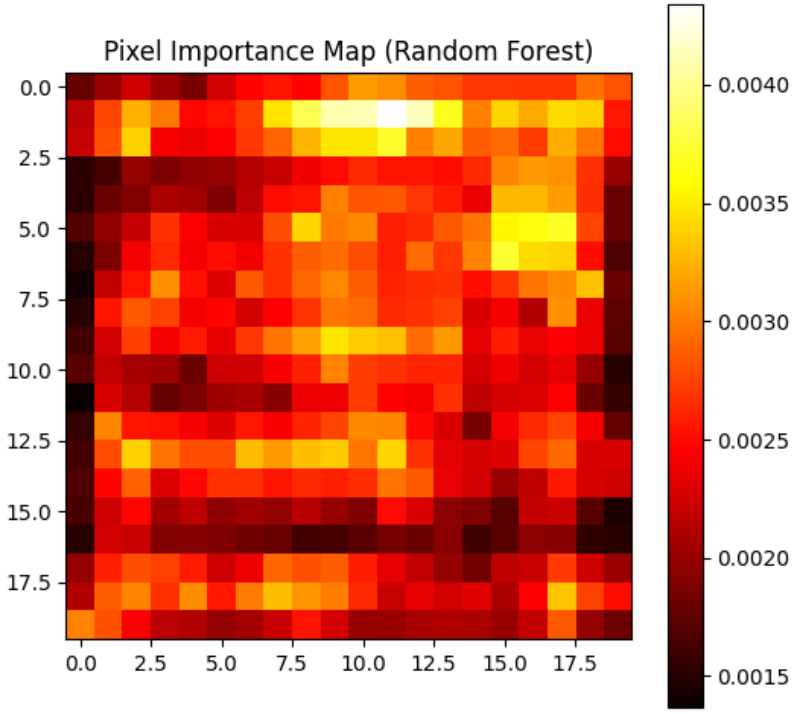


Figure 4: Pixel feature importances

Vizualizacija pokazuje da najveci znacaj imaju pikseli koji se nalaze u centralnim i gornjim regionima slike. Ovi regioni najcesce sadrze kljucne poteze karaktera, kao sto su vertikalne i horizontalne linije i krivine, koji nose najvise diskriminativne informacije prilikom razlikovanja klasa.

Nasuprot tome, pikseli na samim ivicama slike imaju znatno manju vaznost, sto je ocekivano jer u tim delovima slike u vecini slucajeva dominira pozadina bez znacajnih informacija.

Takodje se moze primetiti da vaznost piksela nije rasporedjena potpuno uniformno po regionima karaktera, vec se cesto pojavljuju pojedinacni pikseli sa vecom vaznoscu u odnosu na susedne. Ovakav obrazac je tipican za modele zasnovane na odlukama stabala. Random Forest ne analizira direktno geometrijske strukture poput linija ili oblika, vec pojedinacne atribute. Ukoliko je odredjeni piksel dovoljno informativan za razdvajanje klasa, model ce ga cesto koristiti u cvorovima stabla, dok susedni pikseli koji nose slicnu informaciju mogu biti korisceni redje. Zbog toga mapa vaznosti moze imati "tackasti" karakter, gde pojedinacni pikseli predstavljaju reprezentativne pokazatelje vaznih regiona slike.

Ova analiza potvrdjuje da model uspesno identifikuje delove slike koji nose najvise informacija o strukturi karaktera. Najvazniji atributi nalaze se upravo u regionima gde se tipicno pojavljuju potezi karaktera, sto ukazuje da model uci relevantne vizuelne obrasce, a ne nasumicne sumove u podacima.

Napomena: Pogledati plot u src/models/randomforest.ipynb.

6.5 PCA redukcija

Kod Random Forest modela primena PCA cesto ne donosi poboljsanje performansi, a u nekim slucajevima moze dovesti i do pogorsanja rezultata. Razlog za to lezi u nacinu rada stabala odlucivanja. Random Forest gradi veliki broj stabala koja u svakom cvoru biraju jedan atribut i prave podelu na osnovu jednostavnog uslova oblika $x_j < t$. Kada se koristi originalni prostor atributa, takve podele mogu direktno odgovarati znacajnim lokalnim osobinama podataka. Nakon primene PCA atributi postaju linearne kombinacije velikog broja originalnih osobina, pa podele koje stabla pronalaze vise ne odgovaraju jednostavnim i intuitivnim granicama u prostoru podataka.

Model	Accuracy	Macro F1	Weighted F1
Random Forest	0.708096	0.60042	0.697774
Random Forest + PCA	0.633306	0.52480	0.615194

Table 3: Poredjenje performansi Random Forest modela sa i bez primene PCA.

Rezultati eksperimenta pokazuju da primena PCA dovodi do primetnog pogorsanja performansi Random Forest modela. Model treniran nad originalnim preprocesiranim podacima ostvaruje tacnost od 0.708096, makro F1 meru 0.60042 i ponderisanu F1 meru 0.697774. Nakon primene PCA tacnost opada na 0.633306, makro F1 mera na 0.52480, dok ponderisana F1 mera iznosi 0.615194. Ovaj pad performansi je znatno izrazeniji nego kod prethodno analiziranih modela. Takav rezultat ukazuje na to da su mnoge diskriminativne informacije za Random Forest bile prisutne u originalnim atributima, dok su projekcijom u manji broj PCA komponenti te informacije delom izgubljene.

Zbog toga redukcija dimenzionalnosti pomocu PCA u ovom slucaju ne donosi prednost, vec dovodi i do znatnog pogorsanja performansi i sporijeg treninga modela.

7 Naive Bayes

7.1 Koncept

Naive Bayes predstavlja klasu probabilistickih klasifikacionih algoritama zasnovanih na Bayes-ovoj teoremi. Osnovna ideja ovog pristupa jeste da se verovatnoca pripadnosti uzorka odredjenoj klasi racuna na osnovu uslovnih verovatnoca vrednosti atributa datog uzorka. Model pretpostavlja da su atributi medjusobno nezavisni kada je klasa poznata, sto predstavlja tzv. naivnu pretpostavku nezavisnosti. Iako je ova pretpostavka u praksi cesto pojednostavljenje stvarnih odnosa u podacima, Naive Bayes modeli cesto postižu iznenadjujuce dobre performanse u mnogim klasifikacionim problemima.

Formalno, cilj modela je da za dati uzorak $x = (x_1, x_2, \ldots, x_n)$ odredi klasu y koja maksimizuje posteriornu verovatnocu $P(y \mid x)$. Na osnovu Bayes-ove teoreme ova verovatnoca moze se zapisati kao

$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)}.$$

Kako je imenilac $P(x)$ isti za sve klase, klasifikacija se svodi na maksimizaciju izraza $P(x \mid y)P(y)$. Uz pretpostavku uslovne nezavisnosti atributa, zajednicka verovatnoca $P(x \mid y)$ moze se predstaviti kao proizvod pojedinacnih verovatnoca atributa

$$P(x \mid y) = \prod_{i=1}^{n} P(x_i \mid y).$$

Na taj nacin klasifikacija se svodi na izbor klase koja maksimizuje proizvod verovatnoca pojedinacnih atributa i priorne verovatnoce klase.

U okviru ovog projekta za implementaciju algoritma koriscena je klasa `GaussianNB` iz biblioteke *scikit-learn*. Ova implementacija pretpostavlja da su vrednosti atributa unutar svake klase raspodeljene prema normalnoj (Gaussian) raspodeli. Tokom procesa treniranja model procenjuje srednju vrednost i varijansu svakog atributa za svaku klasu, kao i verovatnoce klasa. Na osnovu ovih statistika model je u stanju da izracuna verovatnocu da dati uzorak pripada svakoj od mogucih klasa.

Jedna od glavnih prednosti Naive Bayes algoritma jeste njegova racunska efikasnost. Trening modela svodi se na racunanje osnovnih statistika nad skupom podataka, zbog cega je vreme treniranja linearno u odnosu na broj uzoraka i broj atributa. Zbog toga je ovaj algoritam posebno pogodan za rad sa velikim skupovima podataka, poput dataset-a koji se koristi u ovom projektu. Iako pojednostavljujuce pretpostavke modela mogu ograniciti njegovu sposobnost modelovanja slozenih zavisnosti izmedju atributa, Naive Bayes cesto predstavlja veoma dobar pocetni model i referentnu tacku za poredjenje sa slozenijim klasifikacionim metodama.

7.2 Preprocesiranje podataka

Za razliku od nekih drugih klasifikacionih algoritama koji zahtevaju dodatne korake transformacije ulaznih podataka, u slucaju Naive Bayes klasifikatora nije bilo neophodno primenjivati posebne metode preprocesiranja. Razlog za to lezi u samoj prirodi algoritma i nacinu na koji on modeluje podatke.

U ovom radu koriscena je varijanta Gaussian Naive Bayes algoritma, koja pretpostavlja da su vrednosti svakog atributa unutar date klase raspodeljene prema normalnoj raspodeli. Tokom procesa treniranja model procenjuje srednju vrednost i varijansu svakog atributa za svaku klasu. Na osnovu tih statistika kasnije se racunaju verovatnoce pripadnosti novog uzorka odredjenoj klasi.

Za razliku od algoritama kao sto su logisticka regresija ili SVM, koji su osetljivi na skalu atributa i cesto zahtevaju standardizaciju ili normalizaciju podataka, Naive Bayes ne koristi optimizacione procedure niti distance izmedju uzoraka. Umesto toga, model direktno procenjuje statisticke parametre raspodele atributa. Zbog toga promena skale atributa u praksi nema znacajan uticaj na rad modela.

U dataset-u koji se koristi u ovom projektu svi atributi predstavljaju numericke vrednosti piksela i nalaze se u istom opsegu vrednosti. Takodje, podaci ne sadrze nedostajuce vrednosti. Iz tih razloga dodatni koraci preprocesiranja, poput standardizacije, normalizacije ili imputacije nedostajucih vrednosti, nisu bili neophodni. Model je treniran direktno nad originalnim vrednostima atributa.

7.3 Konstrukcija modela

Za implementaciju Naive Bayes klasifikatora koriscena je klasa `GaussianNB` iz biblioteke *scikit-learn*. Ova implementacija predstavlja varijantu Naive Bayes algoritma namenjenu radu sa kontinuiranim numerickim atributima, pri cemu se pretpostavlja da su vrednosti atributa unutar svake klase raspodeljene prema normalnoj raspodeli.

Za razliku od mnogih drugih klasifikacionih algoritama, Naive Bayes poseduje veoma mali broj hiperparametara. Tokom procesa treniranja model automatski procenjuje sve potrebne statisticke parametre na osnovu trening podataka. Konkretno, za svaki atribut i svaku klasu izracunavaju se srednja vrednost i varijansa, dok se priorne verovatnoce klasa procenjuju na osnovu relativne ucestalosti klasa u trening skupu.

7.4 Rezultati modela

Performanse modela evaluirane su na test skupu koriscenjem standardnih metrika za klasifikaciju: tacnost (accuracy), makro F1 mera (macro F1) i ponderisana F1 mera (weighted F1). Ove metrike omogucavaju uvid kako u ukupnu tacnost modela, tako i u njegovu sposobnost da pravilno klasifikuje razlicite klase.

Tacnost modela iznosi 0.313, sto znaci da je model uspesno klasifikovao priblizno 31.3% uzoraka iz test skupa. Ovakav rezultat je donekle ocekivan, imajuci u vidu slozenost zadatka, ali i dve osnovne pretpostavke modela: nezavisnost atributa i normalnu raspodelu vrednosti atributa.

Makro F1 mera iznosi 0.167. Relativno niska vrednost ove mere ukazuje da model ima poteskoce pri klasifikaciji pojedinih klasa, posebno onih koje su slabije zastupljene u dataset-u. Ponderisana F1 mera iznosi 0.311.Vrednost ove metrike bliska tacnosti pokazuje da model relativno bolje klasifikuje dominantne klase u dataset-u.

Dobijeni rezultati ukazuju da Naive Bayes klasifikator pruza osnovni nivo performansi nad datim skupom podataka, ali pretpostavka nezavisnosti atributa ogranicava sposobnost modela da uhvati kompleksne zavisnosti izmedju piksela koji opisuju karaktere.

Kao i kod prethodnih modela, analizirane su najcesce konfuzije izmedju parova karaktera. U tabeli su prikazani parovi karaktera kod kojih model najcesce pogresno klasifikuje jedan karakter kao drugi. Rezultati pokazuju da se najveci broj gresaka javlja izmedju vizuelno slicnih karaktera. Najizrazenija konfuzija javlja se izmedju karaktera `1` i `/`, gde je zabelezeno 956 pogresnih klasifikacija. Ovakva greska je ocekivana i "poznata" tj imali smo priliku da je sretnemo u ovoj analizi i kod prethodnih modela. Neke od ostalih gresaka: `0` i `O`, `1` i `9`, `1` i `I`. `6` i `ó`, kao i `4` i `A`.

Generalno posmatrano, analiza konfuzija pokazuje da Naive Bayes klasifikator najcesce pravi greske izmedju karaktera koji su vizuelno veoma slicni. Ovo je ocekivano s obzirom na pretpostavku nezavisnosti atributa koju ovaj model koristi. Naime, Naive Bayes posmatra svaki piksel nezavisno od ostalih, zbog cega nije u stanju da u potpunosti modeluje slozene prostorne odnose izmedju piksela koji cesto nose kljucne informacije o obliku karaktera. Ono sto se vec moze proglasiti ucestalim, svi modeli (do sada) prave slicne greske cesto - ovo govori da problem ne lezi samo u njima samima, vec i u kompleksnosti zadatka.

Kako bi se dobio detaljniji uvid u ponasanje modela, analizirana je tacnost klasifikacije za svaku pojedinacnu klasu karaktera. Na osnovu rezultata identifikovani su karakteri koji su najlosije, kao i oni koji su najbolje klasifikovani. Analiza pokazuje da postoji nekoliko karaktera za koje model nije uspeo da napravi nijednu tacnu klasifikaciju. Medju najlosije klasifikovanim karakterima nalaze se `\x00`, `l`, `x`, `y` i `l` (sa dijakritikom), kao i nekoliko karaktera iz prosirenog ASCII opsega poput `\x81`, `\x84`, `\x86`, `\x87` i `\x88`. Za ove karaktere tacnost klasifikacije iznosi 0%, sto znaci da model nije uspeo da ispravno prepozna nijedan primer iz test skupa. Ovo nam govori da nijedan model (ponovo, do sada) ne uspeva da ih klasifikuje nijednom tacno - problem je definitivno u njima.

Sa druge strane, identifikovani su i karakteri koje model relativno dobro klasifikuje. Najbolje klasifikovani karakteri su `<`, `/`, `7`, `>`, `3`, `(`, `$`, `P` i `L`. Najvecu tacnost pokazuje karakter `<` sa oko 80% ispravnih klasifikacija, zatim `/` sa oko 78% i `7` sa oko 73%.

Interesantno je primetiti da su najbolje klasifikovani karakteri uglavnom oni koji imaju relativno jednostavnu i prepoznatljivu strukturu, dok se medju najlosije klasifikovanim pojavljuju karakteri iz prosirenog skupa ili karakteri koji se verovatno javljaju redje u datasetu. Ovakva analiza nam omogucava bolji uvid u to koje vrste karaktera model najlakse prepoznaje, a kod kojih dolazi do najveceg broja gresaka.

U cilju detaljnije analize performansi modela, ispitana je tacnost klasifikacije za svaki pojedinacni font u datasetu. Rezultati pokazuju znacajne razlike u uspesnosti klasifikacije izmedju razlicitih tipova fontova.

U ovom modelu, najbolji rezultati postignuti su za fontove `E13B`, `OCRB`, `NUMERICS` i `OCRA`, gde tacnost prelazi 75%, dok za font `E13B` dostize oko 85%. Sa druge strane, znacajno losiji rezultati primeceni su kod fontova `YI BAITI`, `CALIBRI`, `TAHOMA`, `SCRIPT` i `SERIF`, gde tacnost pada na vrednosti izmedju 3% i 5%.

Kao i kod prethodnih modela, ocekivano ponasanje je potvrdjeno kada su u pitanju italic i strength parametri podataka - model najbolje radi za neiskosene (italic = 0) karaktere srednje debljine (strength 0.4 - 0.6). Zanimljivo, accuracy procenti u ove 2 kategorije su skoro identicni (40

7.5 Analiza sigurnosti predikcija

Pored standardnih metrika performansi, sprovedena je dodatna analiza sigurnosti predikcija koje model generise. Cilj ove analize bio je da se ispita koliko je model siguran u svoje odluke i da li postoji razlika u sigurnosti izmedju tacnih i pogresnih klasifikacija.

Za svaki uzorak iz test skupa model `GaussianNB` omogucava izracunavanje posteriornih verovatnoca pripadnosti svim klasama koriscenjem funkcije `predict_proba`. Na taj nacin za svaki uzorak dobijen je vektor verovatnoca cija je duzina jednaka broju klasa. Najveca vrednost u tom vektoru predstavlja verovatnocu klase koju je model izabrao kao predikciju i interpretira se kao mera sigurnosti modela u datu odluku.

Na osnovu ovih vrednosti formirana su dva skupa podataka. Prvi skup sadrzi sigurnosti predikcija za uzorke koji su tacno klasifikovani, dok drugi skup sadrzi sigurnosti za uzorke kod kojih je model napravio gresku. Za oba skupa izracunata je prosecna vrednost sigurnosti predikcije.

Rezultati pokazuju da prosecna sigurnost modela za tacno klasifikovane uzorke iznosi približno 0.988, dok za pogresno klasifikovane uzorke iznosi oko 0.940. Iako je sigurnost nesto visa kod tacnih predikcija, razlika izmedju ove dve vrednosti je relativno mala. Ovakav rezultat ukazuje da model cesto dodeljuje veoma visoke verovatnoce svojim predikcijama cak i u situacijama kada pravi gresku.

Ovakvo ponasanje predstavlja poznatu karakteristiku Naive Bayes klasifikatora i posledica je osnovne pretpostavke ovog modela o uslovnoj nezavisnosti atributa. Kao sto smo vec pomenuli, model pretpostavlja da su svi atributi nezavisni kada je klasa poznata, sto omogucava da se zajednicka verovatnoca atributa izracuna kao proizvod verovatnoca pojedinacnih atributa. Formalno, ova pretpostavka moze se zapisati kao

$$P(x \mid y) = \prod_{i=1}^{n} P(x_i \mid y).$$

U dataset-u koji se koristi u ovom radu svaki uzorak je opisan velikim brojem atributa (pikselima karaktera). Prilikom racunanja posteriorne verovatnoce klase model mnozi veliki broj verovatnoca pojedinacnih atributa. I male razlike u verovatnocama izmedju klasa mogu se zbog ovog mnozenja znacajno uvecati, sto dovodi do toga da model generise veoma visoke posteriorne verovatnoce za jednu klasu u odnosu na ostale.

Zbog toga Naive Bayes cesto proizvodi predikcije koje su veoma samouverene, cak i kada su pogresne. Drugim recima, model nema dobar mehanizam za procenu sopstvene nesigurnosti, jer pretpostavka nezavisnosti atributa dovodi do preteranog pojacavanja verovatnoca tokom racunanja posteriora. Ova pojava je poznata u literaturi kao losa kalibracija verovatnoca (*poor probability calibration*) i predstavlja jedno od ogranicenja Naive Bayes pristupa.

7.6 PCA redukcija

Rezultati dobijeni treniranjem Naive Bayes modela nad originalnim skupom atributa i nad PCA-redukovanim skupom prikazani su u Tabeli 4.

Model	Accuracy	Macro F1	Weighted F1
Naive Bayes	0.313	0.167	0.311
Naive Bayes + PCA	0.395	0.272	0.400

Table 4: Poredjenje performansi Naive Bayes modela sa i bez PCA redukcije dimenzionalnosti

Iz tabele se moze primetiti da primena PCA transformacije dovodi do znacajnog poboljsanja performansi modela. Tacnost klasifikacije se povecava sa oko 31% na priblizno 39.5%, sto predstavlja poboljsanje od skoro 8 procentnih poena. Slican trend moze se uociti i kod F1 mera, gde i *Macro F1* i *Weighted F1* beleze vidljiv rast.

Razlog za ovakav pomak moze se objasniti pretpostavkama na kojima se zasniva Naive Bayes algoritam. Ovaj model pretpostavlja da su atributi medjusobno nezavisni. U praksi ova pretpostavka retko potpuno vazi, ali sto su atributi manje korelisani, to model obicno daje bolje rezultate.

U originalnom skupu podataka atributi predstavljaju vrednosti piksela slike. Pikseli koji se nalaze blizu jedni drugih na slici cesto imaju slicne vrednosti, sto znaci da su mnogi atributi medjusobno korelisani. Takva korelacija direktno narusava osnovnu pretpostavku nezavisnosti koju koristi Naive Bayes model i moze negativno uticati na procenu verovatnoca koje model koristi pri klasifikaciji.

Primena PCA transformacije menja reprezentaciju podataka. Novi atributi, odnosno glavne komponente, konstruisani su tako da budu medjusobno ortogonalni, odnosno nekorelisani. Zbog toga su atributi u PCA prostoru znatno manje korelisani nego u originalnom prostoru piksela. Na taj nacin pretpostavka nezavisnosti koju koristi Naive Bayes model u vecoj meri vazi, sto modelu omogucava da napravi preciznije procene i time poboljsa rezultate klasifikacije.

Kako je vreme treninga bez PCA optimizacije bilo svakako vrlo kratko, ne vidimo veliku razliku u duzini trajanja treninga. Trening nad neredukovanim podacima trajao je svega 2 sekunde, dok je trening nad PCA podacima bio jos kraci, 0.85 sekundi. Razlika nije velika, ali je uocljiva i ocekivana.

7.6.1 Analiza sigurnosti predikcija

Dodatna analiza sigurnosti predikcija pokazuje ocekivani efekat nakon primene PCA transformacije nad podacima. Prosecna sigurnost modela za tacno klasifikovane uzorke iznosi 0.90, dok za pogresne predikcije opada na oko 0.63.

Ovaj rezultat ukazuje da smanjenje korelacije izmedju dimenzija samo "pojacava" vazeci status naivne pretpostavke algoritma, odakle i dobijamo bolje rezultate u ovoj specificnoj analizi.

8 Gradient Boosting

8.1 Koncept

Gradient Boosting predstavlja ansambl metodu ucenja koja se zasniva na ideji **sekvencijalnog poboljsavanja modela**. Za razliku od nekih drugih ansambl pristupa (npr. bagging metoda), gde se modeli treniraju nezavisno i paralelno, kod gradient boostinga se modeli treniraju **jedan za drugim**, pri cemu svaki naredni model pokusava da ispravi greske prethodnih modela.

Osnovna ideja algoritma je sledeca: umesto da odmah pokusamo da naucimo slozenu funkciju koja mapira ulazne podatke u klase, model se gradi postepeno kao suma velikog broja jednostavnih modela. Najcesce su ti jednostavni modeli stabla odlucivanja male dubine (tzv. *weak learners*). Svako novo stablo se trenira tako da aproksimira negativni gradijent funkcije gubitka, odnosno da model usmeri u pravcu u kome se greska klasifikacije smanjuje.

Proces ucenja moze se intuitivno opisati kroz sledece korake. Na pocetku se trenira veoma jednostavan model koji daje pocetnu procenu izlaza. Nakon toga se izracunava greska tog modela nad trening skupom. Sledeci model se zatim trenira tako da predvidja upravo te greske, odnosno razliku izmedju stvarnih vrednosti i predikcija trenutnog ansambla. Kada se taj model doda postojecem modelu, ukupna greska se smanjuje. Ovaj postupak se ponavlja kroz vise iteracija, pri cemu svaki novi model doprinosi poboljsanju ukupne predikcije.

Vazan aspekt gradient boostinga je i **parametar stope ucenja (learning rate)** koji kontrolise koliki doprinos svaki novi model ima u ukupnom ansamblu. Manja vrednost ovog parametra znaci da svaki model doprinosi manje, ali cesto vodi ka stabilnijem ucenju i boljoj generalizaciji, uz potrebu za vecim brojem iteracija.

Zahvaljujuci ovom principu iterativnog smanjivanja greske, gradient boosting je sposoban da modeluje veoma slozene odnose u podacima. Zbog toga se ubraja medju najmocnije algoritme masinskog ucenja za **tabularne podatke**, gde cesto postize vrlo visoku tacnost klasifikacije ili regresije. U ovom projektu koristi se optimizovana implementacija iz biblioteke *scikit-learn*, koja omogucava efikasno treniranje cak i nad velikim skupovima podataka.

8.2 Konstrukcija modela

Za implementaciju algoritma korišćena je optimizovana implementacija iz biblioteke *scikit-learn*, konkretno klasa `HistGradientBoostingClassifier`. Ova implementacija koristi histogramski pristup prilikom gradnje stabala, čime se značajno ubrzava trening na većim skupovima podataka u poredjenju sa klasičnim gradient boosting algoritmom.
Model je inicijalizovan sa sledećim parametrima:

```
model = HistGradientBoostingClassifier(
    max_depth = 10,
    max_iter = 120,
    learning_rate = 0.1,
    random_state = 42,
    max_leaf_nodes = 31,
    verbose = 2
)
```

Parametar `max_depth` odredjuje maksimalnu dubinu pojedinačnih stabala odlučivanja. Postavljena vrednost je 10, što omogućava modelu da nauči dovoljno složene obrasce u podacima, ali da pritom ne dodje do prevelike kompleksnosti stabala. Veća dubina bi potencijalno mogla dovesti do preprilagodjavanja modela, ali bi takodje značajno produžila vreme treniranja.

Parametar `max_iter` predstavlja broj iteracija algoritma, odnosno broj stabala koja se sekvencijalno dodaju u ansambl. U ovom slučaju odabrana je vrednost 120, što se pokazalo kao dobar kompromis izmedju kvaliteta modela i vremena treniranja. Veći broj iteracija često može poboljšati performanse, ali takodje linearno povećava trajanje treninga. Model je zavrsio u 11-oj iteraciji sto ostavlja pitanje da li su ostali parametri uskladjeni sa ovim.

Parametar `learning_rate` kontrolise koliki doprinos svako novo stablo ima u ukupnom modelu. Vrednost 0.1 predstavlja cesto koriscenu podrazumevanu vrednost u praksi, jer omogucava stabilno ucenje i postepeno smanjivanje greske modela bez prevelikih oscilacija tokom treninga.

Parametar `random_state` postavljen je na vrednost 42 kako bi se obezbedila reproduktivnost rezultata. Na ovaj nacin se obezbedjuje da se pri svakom pokretanju algoritma dobiju identicni rezultati, sto je vazno za analizu performansi i poredjenje sa drugim modelima.

Parametar `max_leaf_nodes` ogranicava maksimalan broj listova koje jedno stablo moze imati. Postavljena vrednost 31 dodatno kontrolise kompleksnost stabala i sprecava njihovo preterano grananje, cime se smanjuje rizik od prenaucenosti i ubrzava proces treniranja.

Na kraju, parametar `verbose` postavljen je na vrednost 2 kako bi se tokom treninga prikazivale informacije o napretku algoritma. Ovo je bilo korisno za praćenje trajanja treninga i procenu vremena potrebnog za završetak procesa.

S obzirom na to da preciznost nije bila najvisa moguca, razmatrao sam da li je ovaj skup parametara dobar. Sumnjao sam da male vrednosti za `max_leaf_nodes` i learning rate 0.1 mozda "opstruisu" razvitak modela jer se vidi da je zavrsen u 11oj od 120 maksimalnih iteracija. Medjutim, trening modela završen je znatno pre dostizanja maksimalnog broja iteracija jer implementacija algoritma koristi mehanizam ranog zaustavljanja (early stopping). Na osnovu validacionog skupa procenjeno je da dalje dodavanje novih stabala ne dovodi do poboljšanja performansi modela, zbog čega je proces treniranja automatski prekinut.

8.3 Analiza rezultata

Dobijeni rezultati pokazuju da model postiže tačnost od približno 43.5%, što ukazuje na relativno ograničenu uspešnost klasifikacije u odnosu na očekivanja. Posebno je značajno primetiti da je vrednost *Macro F1* mere oko 0.29, što sugeriše da model ima poteškoće da podjednako dobro klasifikuje sve klase, odnosno da mu je dodatno tesko klasifikovati klase koje nisu ucestale u celom skupu.

Sa druge strane, vrednost *Weighted F1* mere iznosi približno 0.43, što je blisko vrednosti tačnosti modela. Ovo ukazuje na to da model bolje klasifikuje klase koje su zastupljenije u skupu podataka, dok performanse na redjim klasama ostaju slabije. Ukupno posmatrano, rezultati pokazuju da primenjeni gradient boosting model nije uspeo da ostvari visoku preciznost na ovom zadatku.

Analiza matrice konfuzije dodatno potvrdjuje ogranicene performanse modela. Moze se primetiti da model cesto mesa cak i relativno jednostavne i vizuelno prepoznatljive karaktere, ukljucujuci osnovne cifre, na primer $9 \rightarrow 4$, $2 \rightarrow 1$ ili $6 \rightarrow 1$.

Ovakav obrazac gresaka sugerise da model nije uspeo da u dovoljnoj meri nauci karakteristicne obrasce u skupu podataka. Drugim recima, naucene granice odlucivanja nisu dovoljno precizne da pouzdano razdvoje pojedine klase karaktera, sto se direktno odrazava na nize vrednosti metrika evaluacije.

Detaljnija analiza tačnosti po klasama dodatno potvrdjuje prethodne zakljucke. Najlosije klasifikovani karakteri imaju veoma nisku tacnost, cesto ispod 10%, sto znaci da model u velikom broju slucajeva pogresno predvidja njihovu klasu. Ovo ukazuje na to da model ima ozbiljne poteskoce u razlikovanju odredjenih karaktera, posebno onih koji se redje pojavljuju ili imaju slozeniju strukturu u odnosu na druge.

Sa druge strane, analiza najbolje klasifikovanih karaktera pokazuje da model znatno bolje prepoznaje pojedine cifre. Na primer, karakteri poput 0, 1, 7, 3 i 6 postizu tacnost izmedju priblizno 74% i 79%. Ovakav rezultat sugerise da model prepoznaje karaktere srazmerno uspesno svojoj generalnoj tacnosti.

Analiza fontova je dala ocekivane rezultate, jednostavniji fontovi su klasifikovani sa priblizno 70 posto uspesnosti dok oni kompleksniji variraju izmedju 5-10 procenata.

Dodatna analiza performansi modela u odnosu na stil karaktera pokazuje jasnu razliku u uspešnosti klasifikacije. Za standardne (neiskosene) karaktere model postize tačnost od priblizno 51.5%, dok za italik karaktere tacnost opada na oko 24.8%. Ovakav rezultat je ocekivan i zastupljen u svim modelima.

Slican obrazac moze se primetiti i prilikom analize u odnosu na jacinu ili izrazenost karaktera. Karakteri srednje jacine (opseg 0.4–0.6) klasifikovani su sa tacnoscu od oko 51.9%, dok kod nesto izrazenijih karaktera (opseg 0.6–0.8) tacnost opada na priblizno 24.7%. Ovo dodatno sugerise da promene u vizuelnom izgledu karaktera znacajno uticu na performanse modela i otezavaju njegovo pouzdano prepoznavanje.

8.4 PCA redukcija

Kod Gradient Boosting modela efekat primene PCA je cesto slican kao kod Random Forest modela. Ovi modeli su zasnovani na stablima odlucivanja i tokom treninga prirodno vrse selekciju atributa birajuci one koji najvise doprinose smanjenju necistoce u cvorovima stabla. Zbog toga veliki broj ulaznih atributa obicno ne predstavlja problem za ovakve modele. Kada se primeni PCA, originalni atributi se zamenjuju linearnim kombinacijama velikog broja osobina, cime se menja struktura prostora u kome stabla pronalaze podele. U takvim uslovima stabla mogu teze identifikovati dobre granice izmedju klasa, sto ponekad dovodi do pogorsanja performansi modela.

Model	Accuracy	Macro F1	Weighted F1
Gradient Boosting	0.434554	0.294551	0.434436
Gradient Boosting + PCA	0.420827	0.282567	0.408115

Table 5: Poredjenje performansi Gradient Boosting modela sa i bez primene PCA.

Rezultati eksperimenta pokazuju upravo takav efekat. Gradient Boosting model treniran nad originalnim preprocesiranim podacima ostvaruje tacnost od 0.434554, makro F1 meru 0.294551 i ponderisanu F1 meru 0.434436. Nakon primene PCA dolazi do blagog pada performansi: tacnost opada na 0.420827, makro F1 mera na 0.282567, dok ponderisana F1 mera iznosi 0.408115. Iako je pad manji nego kod Random Forest modela, sve metrike pokazuju negativan trend, sto ukazuje na to da redukcija dimenzionalnosti uklanja deo informacija koje su bile korisne za klasifikaciju.

Za razliku od Random Forest modela, kod Gradient Boosting algoritma primecuje se znacajno smanjenje vremena treninga nakon primene PCA. Trening modela nad originalnim podacima trajao je oko 903 sekunde, sto odgovara vremenu od priblizno 15 minuta. Nakon redukcije dimenzionalnosti pomocu PCA vreme treninga smanjeno je na oko 236 sekundi, odnosno priblizno 4 minuta. Razlog za ovakvo ubrzanje lezi u cinjenici da Gradient Boosting trenira stabla sekvencijalno, pa smanjenje dimenzionalnosti direktno utice na brzinu izgradnje svakog pojedinacnog stabla u modelu.

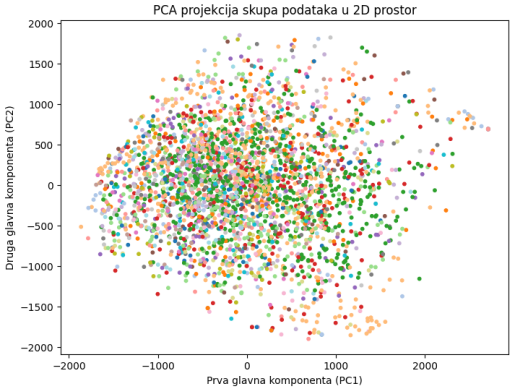
Na osnovu ovih rezultata moze se zakljuciti da primena PCA u ovom slucaju nije dovela do poboljsanja klasifikacionih performansi Gradient Boosting modela, ali je ubrzala proces treninga. Ovakav rezultat ukazuje na kompromis izmedju brzine treniranja i kvaliteta modela, gde redukcija dimenzionalnosti ubrzava ucenje modela, ali moze dovesti do gubitka diskriminativnih informacija prisutnih u originalnom prostoru atributa.

9 Sveukupna analiza i poredjenje performansi modela

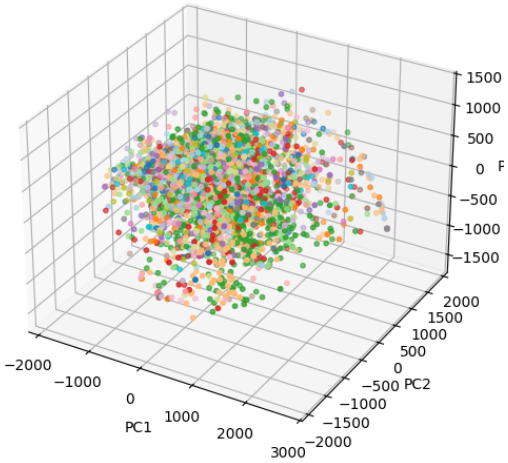
9.1 Vizualizacija podataka

Da bismo sto bolje procenili kvalitet dobijenih modela, moramo se upoznati sa problemom koji resavamo. Vizualizacija podataka moze posluziti kao alat za grubo upoznavanje strukture podataka. S obzirom da nas skup podataka sadrzi tacno 400 dimenzija, nemoguce je predstaviti ga u nekom intuitivnom prostoru. Kada na to dodamo da je broj instanci koje posmatramo (bar u trening skupu) okvirno 667 hiljada, nemoguce je nacrtati ih sve i dobiti pregledan rezultat koji nesto govori o samom skupu.

Radi boljeg razumevanja strukture skupa podataka izvrsena je vizuelizacija podataka pomocu metode PCA. Kako se originalni podaci nalaze u prostoru visoke dimenzionalnosti (400 atributa), direktna vizuelna analiza takvog prostora nije moguca. Zbog toga je skup podataka projektovan u prostor manjih dimenzija koristeci prve dve, odnosno tri glavne komponente, koje objasnjavaju najveci deo varijanse u podacima. Svaka tacka na grafiku predstavlja jedan uzorak iz skupa podataka, dok boja tacke oznacava klasu kojoj taj uzorak pripada.



(a) 2D Visualization



(b) 3D Visualization

Figure 5

Na osnovu dobijenih vizuelizacija moze se primetiti da se tacke razlicitih boja u velikoj meri preklapaju u prostoru projekcije. Ovakav raspored ukazuje da uzorci iz razlicitih klasa cesto imaju slicne vrednosti atributa, pa granice izmedju klasa nisu jasno razdvojene. Kada se klase znacajno preklapaju u prostoru osobina, klasifikacioni algoritmi moraju pronaci slozenije granice odlucivanja kako bi pravilno razdvojili instance.

U pojedinim regionima mogu se uociti lokalne grupe tacaka koje su medjusobno blizu, sto ukazuje na postojanje odredjenih obrazaca u podacima. Ipak, u tim regionima se cesto pojavljuju tacke razlicitih boja, sto znaci da slicnost u prostoru osobina ne garantuje pripadnost istoj klasi. Takva situacija dodatno otezava zadatak klasifikacije, jer se instance razlicitih klasa mogu nalaziti veoma blizu jedna drugoj.

Takodje je vazno naglasiti ogranicenje ovog pristupa. PCA projekcija prikazuje samo prve dve ili tri glavne komponente koje obuhvataju najveci deo varijanse u podacima, dok informacije iz preostalih dimenzija nisu vidljive na grafiku. Zbog toga ovakva vizuelizacija predstavlja prvenstveno kvalitativni alat za analizu strukture podataka.

Na osnovu prikazanih vizuelizacija moze se zakljuciti da dataset poseduje odredjenu strukturu i lokalne grupe slicnih uzoraka, ali i znacajno preklapanje izmedju klasa. Ovakva raspodela instanci u prostoru osobina ukazuje na slozenost problema klasifikacije i u skladu je sa dobijenim rezultatima klasifikacionih modela.

9.2 Poredjenje performansi modela

Na osnovu prikazanih rezultata, model **Random Forest** ostvaruje ubedljivo najbolje performanse. Njegova vrednost *accuracy* iznosi priblizno 0.71, sto je znacajno vise u odnosu na ostale modele. Slede **logistic regression** sa oko 0.48, zatim **gradient boosting** sa oko 0.43, dok **linearni SVM** postize oko 0.41. Najslabiji rezultat ima **Naive Bayes** sa priblizno 0.31. Ovakva razlika ukazuje da Random Forest najefikasnije modeluje strukturu podataka i uspesnije generalizuje u odnosu na ostale algoritme.

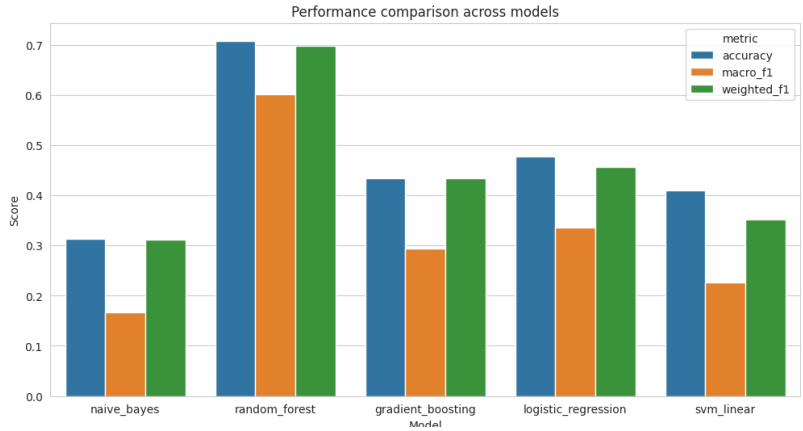


Figure 6: Poredjenje preciznosti modela

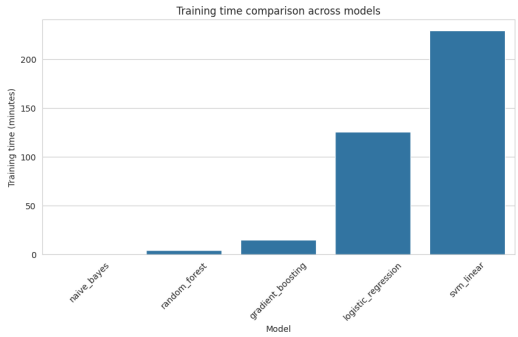
Prednost Random Forest modela moze se objasniti njegovom ansambl prirodom. Kombinovanjem velikog broja stabala odlucivanja model je sposoban da modeluje slozene i nelinearne odnose izmedju karakteristika, sto je posebno vazno u problemu klasifikacije karaktera razlicitih fontova. Agregacija velikog broja stabala smanjuje varijansu pojedinacnih modela i dovodi do stabilnijih predikcija.

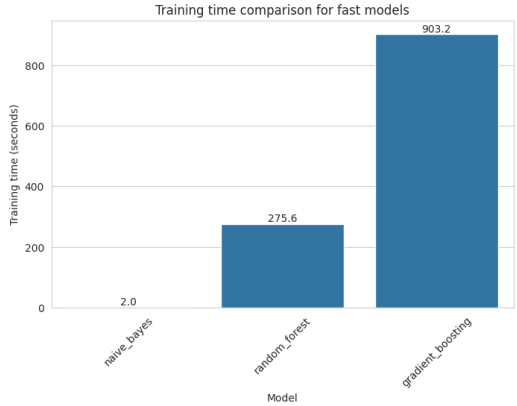
Slican obrazac moze se uociti i kod ostalih metrika. Random Forest ostvaruje i najvise vrednosti *macro F1* (≈ 0.60) i *weighted F1* (≈ 0.70), sto potvrdjuje njegovu superiornost u odnosu na druge pristupe. Kod preostalih modela primecuje se da je *macro F1* znacajno nizi od *weighted F1*, sto ukazuje na neravnomerne performanse izmedju razlicitih klasa. Drugim recima, modeli postizu bolje rezultate na dominantnim klasama, dok imaju vise poteskoca sa redjim ili tezim karakterima.

Ukupno posmatrano, rezultati jasno pokazuju da je **Random Forest** najuspesniji model za ovaj zadatak, dok **logistic regression**, **gradient boosting** i **linearni SVM** ostvaruju srednje, medjusobno relativno slicne performanse. **Naive Bayes** pokazuje najslabije rezultate, sto je ocekivano zbog njegove pretpostavke nezavisnosti karakteristika koja u ovom problemu nije ispunjena.

9.3 Vreme treniranja modela

Pored prediktivnih performansi, analizirano je i vreme treniranja svakog modela. Na osnovu prikazanih rezultata moze se uociti znacajna razlika u racunskoj slozenosti izmedju posmatranih algoritama. Prva analiza prikazuje ukupno vreme treniranja svih modela izrazeno u minutima, kako bi se dobio opsti pregled njihove racunske zahtevnosti.
Najduze vreme treniranja ima **linearni SVM**, kome je potrebno priblizno 230 minuta da zavrsi proces ucenja. Odmah iza njega nalazi se **logistic regression** sa oko 125 minuta. Razlog za ovako dugo vreme treniranja lezi u prirodi ovih algoritama, koji zahtevaju iterativnu optimizaciju nad velikim brojem parametara i prolaze vise puta kroz ceo skup podataka kako bi konvergirali ka optimalnom resenju. Posebno kod linearnih modela treniranih nad velikim brojem atributa, optimizacioni postupak moze biti racunski veoma zahtevan.





Sa druge strane, **gradient boosting** zahteva znatno manje vremena (oko 15 minuta), dok je **random forest** jos brzi sa oko 5 minuta. Iako su oba algoritma zasnovana na ansamblu stabala odlucivanja, proces treniranja pojedinacnog stabla je relativno brz, a u slucaju random forest pristupa stabla se mogu graditi nezavisno jedno od drugog, sto dodatno ubrzava proces treniranja.

Najbrzi model je **Naive Bayes**, cije treniranje traje svega oko 2 sekunde. Ova izuzetna brzina proizilazi iz vrlo jednostavne strukture modela i cinjenice da algoritam ne zahteva slozenu optimizaciju, vec samo procenu osnovnih statistickih parametara distribucije podataka.

Zbog velike razlike izmedju sporih i veoma brzih modela, uradjena je i dodatna analiza koja izdvaja samo najbrze algoritme: **Naive Bayes**, **Random Forest** i **Gradient Boosting**. U ovoj analizi vreme treniranja je prikazano u sekundama kako bi se jasnije uocile razlike izmedju ovih modela. Rezultati pokazuju da je Naive Bayes daleko najbrzi model (oko 2 sekunde), dok Random Forest zahteva oko 276 sekundi, a Gradient Boosting oko 903 sekunde.

9.4 Velicina modela

Na prikazanom grafiku data je uporedna analiza velicine sacuvanih modela. Zbog velikih razlika izmedju modela, vertikalna osa je predstavljena u logaritamskoj skali, sto omogucava jasnije poredjenje modela cije se velicine razlikuju za vise redova velicine. Najveci model je **Random Forest**, cija velicina iznosi priblizno 9000 MB. Ovakva velicina je ocekivana jer Random Forest predstavlja ansambl velikog broja stabala odlucivanja, gde svako stablo mora biti sacuvano u modelu zajedno sa svim granama i parametrima.
Sa druge strane, **Gradient Boosting** model ima znatno manju velicinu, oko 10.5 MB. Iako i ovaj algoritam koristi ansambl stabala, broj i struktura stabala su obicno manji nego kod Random Forest pristupa, a stabla su cesto plitkija, sto znacajno smanjuje memorijski otisak modela.

Model **Naive Bayes** ima velicinu oko 1.6 MB, dok je **Logistic Regression** jos kompaktniji sa svega oko 0.4 MB. Razlog za ovako malu velicinu lezi u prirodi ovih algoritama. Za razliku od ansambl metoda, ovi modeli cuvaju samo skup tezina ili osnovne statisticke parametre distribucija, bez potrebe za skladistenjem velikog broja struktura poput stabala.

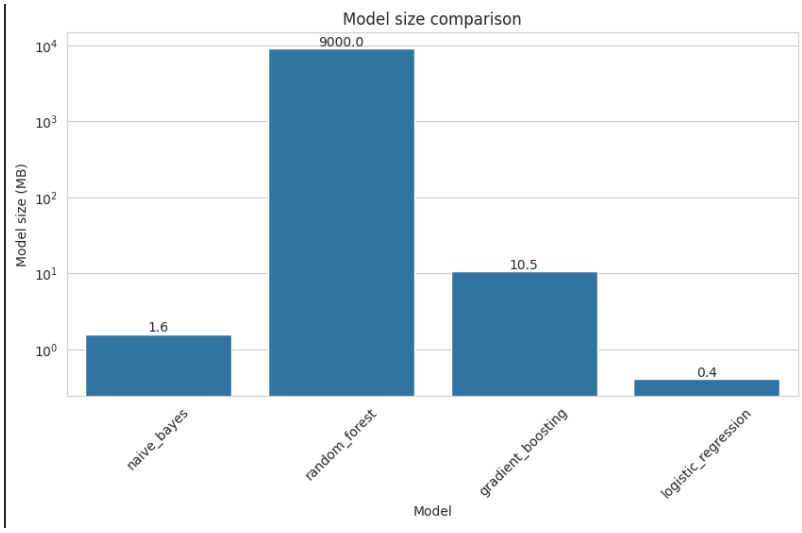


Figure 7: Model sizes

Ukupno posmatrano, rezultati jasno pokazuju znacajnu razliku u memorijskoj slozenosti izmedju modela. Dok ansambl metode, posebno Random Forest, mogu postici bolje performanse, to dolazi uz cenu znatno vece memorijske potrosnje. Sa druge strane, jednostavniji modeli poput Logistic Regression ili Naive Bayes imaju veoma malu memorijsku velicinu, ali po pravilu ostvaruju i slabije klasifikacione performanse.

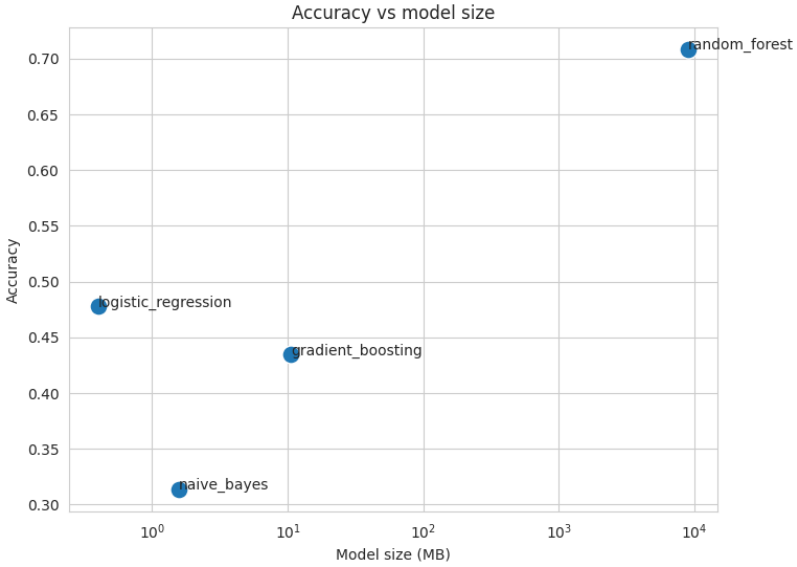


Figure 8: Size vs accuracy

Na prikazanom grafiku predstavljen je odnos izmedju velicine modela i ostvarene tacnosti klasifikacije. Moze se uociti da **Random Forest** ostvaruje najvecu tacnost (oko 0.71), ali uz izuzetno veliku velicinu modela od oko 9000 MB.
Sa druge strane, modeli **Logistic Regression**, **Gradient Boosting** i **Naive Bayes** imaju znatno manju velicinu modela, ali i slabije performanse. Logistic Regression i Gradient Boosting ostvaruju umerenu tacnost uz relativno malo memorijsko zauzece, dok Naive Bayes ima najmanju velicinu modela i najnizu tacnost. Ovi rezultati jasno pokazuju kompromis izmedju velicine modela i njegovih performansi; kompleksniji modeli mogu postici bolje rezultate, ali uz vecu memorijsku cenu.

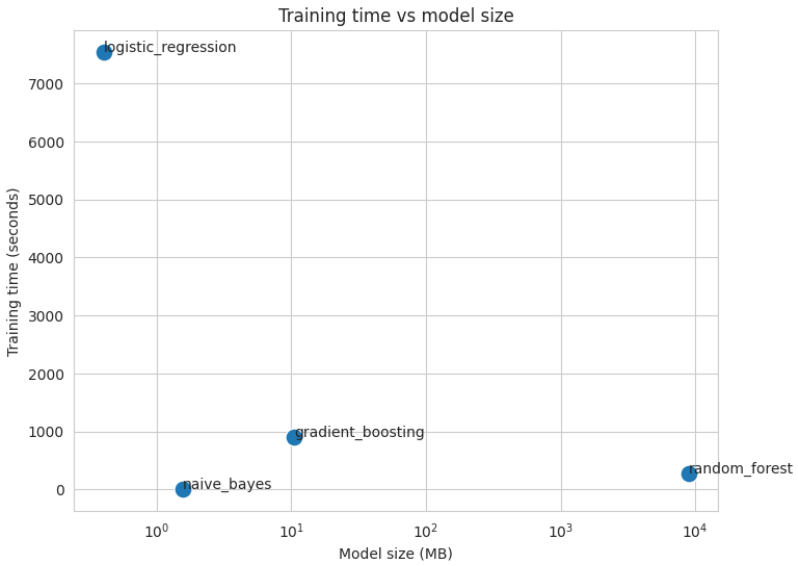


Figure 9: Size vs training time

Na prikazanom grafiku predstavljen je odnos izmedju velicine modela i vremena potrebnog za treniranje. Moze se uociti da **Random Forest**, iako ima ubedljivo najvecu velicinu modela (oko 9000 MB), ima relativno kratko vreme treniranja u odnosu na neke znatno manje modele. Ovo ukazuje na vaznu osobinu ovog algoritma: stabla u Random Forest ansamblu mogu se graditi nezavisno jedno od drugog, sto omogucava efikasno paralelno treniranje i znacajno ubrzava proces ucenja.
Sa druge strane, **Logistic Regression** ima veoma malo memorijsko zauzece, ali zahteva najvise vremena za treniranje. Razlog za to lezi u iterativnoj optimizaciji parametara, gde algoritam mora vise puta prolaziti kroz skup podataka dok ne konvergira ka optimalnom resenju. **Gradient Boosting** zauzima srednju poziciju, jer se stabla treniraju sekvencijalno, pa proces ne moze biti paralelizovan kao kod Random Forest pristupa. **Naive Bayes** ostaje najbrzi model zbog svoje jednostavne statisticke strukture i vrlo malog broja parametara koje je potrebno proceniti. Pored toga sto je najbrzi, on je i memorijski najmanji, ali to sve dolazi po ceni kvaliteta.

9.5 PCA redukcija

Na prikazanom grafiku uporedjene su vrednosti tacnosti klasifikacije pre i nakon primene PCA redukcije dimenzionalnosti. Kao sto je vec ranije primeceno kroz numericke rezultate, primena PCA u vecini slucajeva dovodi do blagog smanjenja tacnosti modela. Ovo je posebno vidljivo kod modela **Random Forest**, **Gradient Boosting**, **Logistic Regression** i **SVM**, gde se nakon redukcije dimenzionalnosti uocava mali pad u vrednosti accuracy.

Sa druge strane, jedini model kod koga dolazi do poboljsanja performansi je **Naive Bayes**. Kod ovog modela primena PCA dovodi do primetnog rasta tacnosti, sto je verovatno posledica uklanjanja redundantnih i korelisanih atributa. Kako Naive Bayes pretpostavlja nezavisnost karakteristika, smanjenje dimenzionalnosti i eliminacija zavisnosti izmedju atributa mogu doprineti boljem uklapanju modela.

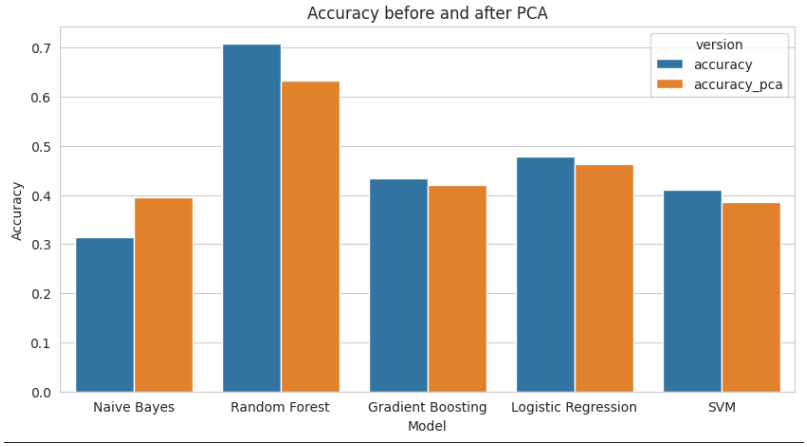


Figure 10: Size vs training time

Iako su ovakvi zakljucci vec mogli biti uoceni kroz tabelarne rezultate, vizuelni prikaz na grafiku dodatno olaksava interpretaciju i jasno pokazuje da PCA u ovom eksperimentu uglavnom ne doprinosi poboljsanju performansi vecine modela, osim u slucaju Naive Bayes algoritma.

9.6 Poredjenje modela kod italic stila

Na prikazanom grafiku uporedjene su performanse modela na standardnim i italic varijantama fontova. Moze se jasno uociti da svi modeli ostvaruju znacajno bolje rezultate na standardnim fontovima nego na italic varijantama. Ovaj pad performansi ukazuje na to da promene u stilu fonta predstavljaju dodatnu tezinu za modele, jer oblik karaktera postaje manje slican uzorcima na kojima su modeli trenirani. I pored toga, **Random Forest** i u ovom scenariju ostvaruje najbolje rezultate. Iako dolazi do pada tacnosti kod italic fontova, ovaj model i dalje zadrzava najvecu vrednost accuracy u odnosu na ostale algoritme, sto pokazuje da ima najbolju sposobnost generalizacije na nove stilove karaktera. Zanimljivo je da ovaj model ima toliku prednost u kvalitetu nad ostalima da mu preciznost nad iskosenim karakterima parira preciznosti ostalih modela nad standardnim, neiskosenim verzijama karaktera.

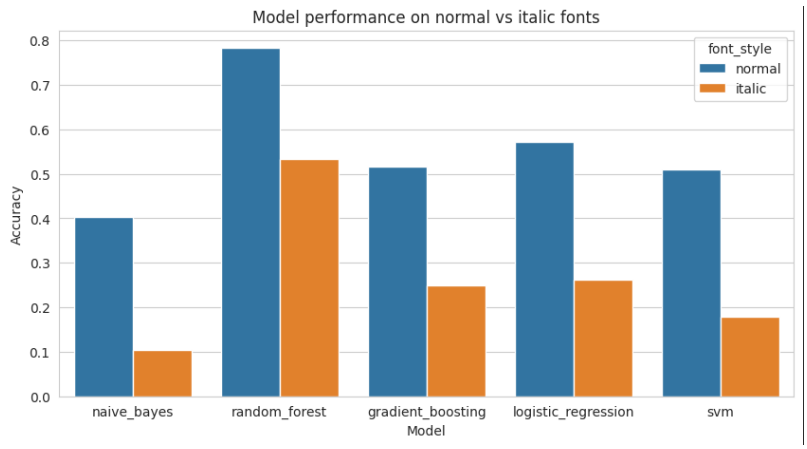


Figure 11: Size vs training time

Modeli **Gradient Boosting**, **Logistic Regression** i **SVM** ostvaruju srednje rezultate, pri cemu se kod svih njih primecuje primetan pad tacnosti na italic fontovima. Najveci pad performansi

moze se uociti kod modela **Naive Bayes**, koji vec ima relativno nisku tacnost na standardnim fontovima, dok na italic varijantama dolazi do dodatnog znacajnog pogorsanja rezultata.
Ukupno posmatrano, rezultati pokazuju da promena stila fonta znacajno utice na performanse svih modela. Iako nijedan model nije potpuno robustan na ovu transformaciju, Random Forest pokazuje najbolju otpornost na promene u vizuelnom obliku karaktera.

9.7 Analiza fontova

Na prikazanom grafiku predstavljeni su fontovi koji su u proseku najuspesnije i najlosije klasifikovani kroz sve posmatrane modele. Rezultati su dobijeni agregacijom tacnosti klasifikacije po fontovima i izdvajanjem pet fontova sa najvecim i pet sa najmanjim prosecnim vrednostima accuracy. Ovakav prikaz omogucava uvid u to koji stilovi karaktera predstavljaju najmanji, odnosno najveci izazov za modele.
Moze se uociti da su fontovi **E13B**, **NUMERICS**, **OCRB**, **VIN** i **OCRA** najlakse klasifikovani, sa znacajno vecim prosecnim vrednostima tacnosti u odnosu na ostale fontove. Ovi fontovi imaju relativno jednostavne i jasno definisane oblike karaktera, sa manjim brojem stilskih varijacija, sto olaksava modelima da izdvoje stabilne vizuelne obrasce.
Sa druge strane, fontovi **YI BAITI**, **SKETCHFLOW**, **EDWARDIAN**, **SCRIPT** i **SERIF** ostvaruju najnize vrednosti tacnosti. Razlog tome moze biti visestruk - zavisi kako su slike generisane za ove fontove. Takodje, zavisi i kojom debljinom su karakteri ispisani kao i to da li su vecinski iskoseni ili ne.

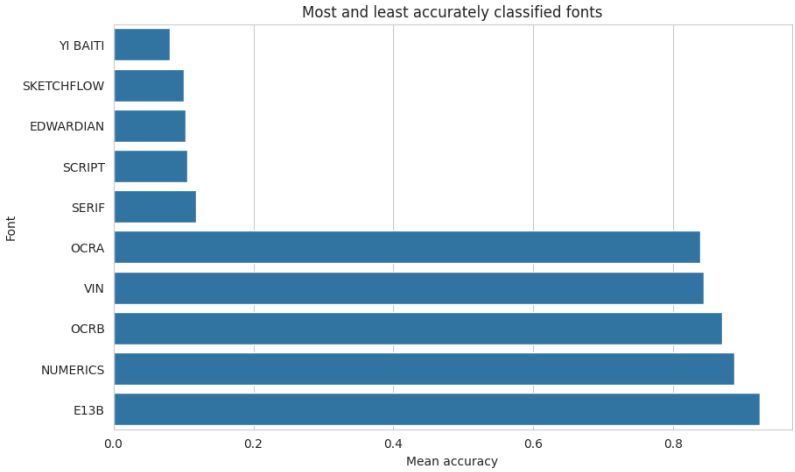
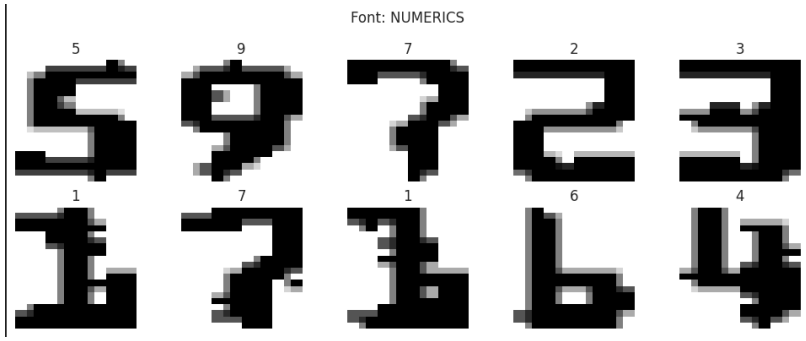
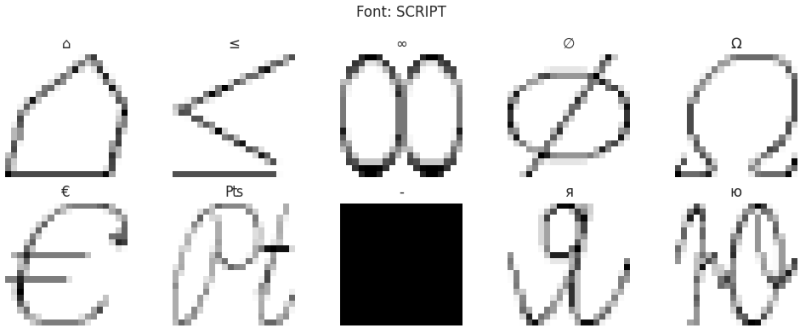


Figure 12: Najlakse i najteze klasifikovani fontovi

Ovi rezultati ukazuju da struktura i stil fonta imaju znacajan uticaj na tezinu problema klasifikacije. Dok fontovi sa jednostavnijim i standardizovanim oblicima karaktera omogucavaju visoku tacnost, stilizovani i dekorativni fontovi predstavljaju znatno veci izazov za modele. Vazno je napomenuti da ne mozemo naivno proglasiti losije klasifikovane fontove kompleksnijim vizuelno, jer ovi rezultati usko zavise i od procenta zastupljenosti "bledih" i iskosenih uzoraka karaktera u pojedinacnim fontovima.

Ilustracija pomenutih fontova: na slikama ispod ilustrovani su nasumicni karakteri iz 2 fonta iznad: Numerics i Script sa ciljem da imamo uvid u njihov stil.





Iz ovog nasumicnog primera mozemo zakljuciti i zasto je Numerics drasticno bolje klasifikovan u odnosu na SCRIPT. Script karakteri su mahom iskoseni ali i nestandardni vizuelno. Vidimo da je svaki karakter iskosen ali mnogi i dodatno ukraseni kvakicama i kosim linijama na krajevima. Ova karakteristika znatno otezava klasifikaciju. Sa druge strane, zanimljivo je da su NUMERICS karakteri vrlo dobro klasifikovani iako vecina njih ima debljinu koja je na pojedinacnim analizama svih modela bila slabije klasifikovana. Ovo je moguce jer karakteri sami po sebi nemaju mnogo ukrasa, najverovatnije nisu iskoseni u velikoj meri i sastoje se od jednostavnih linija, bez previse ukrasa.